

Work Integrated Learning 3A.

Assignment POE Task 2 - E.SOS app: POWA (People Opposing Women Abuse).

Sheketli Mochaki ST10452756
Lerato Molokomme ST1043926
Mulweli Mudau ST10358894
Kgaogelo Mashiane ST10446936

4/30/26

Information Technology in Software Development

Table of Contents

Requirement Analysis 2
1. Introduction 2
1.1. Purpose 2
1.2. Scope 2
1.3. Overview 3
2. Solution Domain 3
2.1. System Environment 3
2.2. User Characteristics 4
3. Functional Requirements 5
3.1. Functional Requirements Master List 5
3.2. Non-Functional Requirements 6
3.3. Detailed Functional Requirement Specifications 6
4. UML Use Case Diagram Background 7
4.1. System Actors 7
4.2. Use Cases 8
4.3. Use Case Relationships 9
5. Logical System Model 10
5.1. System Inputs 10
5.2. System Processes 10
5.3. System Outputs 10
5.4. Data Storage 11
6. Class Diagrams 11
Class Relationships 11
System Design Document 12
7. Introduction 12
8. Logical Architecture Design – High-Level Architectural Design 12
8.1. Architectural Overview 12
8.2. Technology Stack 13
8.3. Security Architecture 13
9. Interaction with User – Input Interactions 14
9.1. Mobile Application – User interaction Flow 14
9.2. Web Dashboard – Operator Interaction Flow 18
9.3. Accessibility Design Standards 24
10. Database Design 24
10.1. Database Design Principles 24
10.2. Database Tables 25
11. Entity Relationship Database Diagram 27
12. System Report Design 28

Mobile App and Website Development (Task 2) 29
1. Introduction 29
2. Project Overview 29
2.1 Background of the Project 29
2.2 Project Objectives 29
3. Project Repositories and Media Links 29
3.1 GitHub Repository Link 29
3.2 Demonstration Video Link 30
4. Mobile Application Interface and Features 30
4.1 User Registration and Login 30
4.2 Single Sign-On (SSO) Implementation 32
4.3 User Settings and Password Management 33
4.4 Help Documentation and User Support 34
5. Database Design and Verification 35
5.1 Relational Database Structure and Tables 35
Screenshots: SQL Table Definitions 37
5.2 Database Population and Verification 39
6 DevOps Lifecycle and Automated Quality Assurance 42
6.1 Kanban Board and Task Management 42
Moving items across the Kanban columns 43
Managing timelines 43
Screenshots 43
6.2 Automated Build Pipeline 46
6.3 Automated Unit Testing and Build Results 46
Appendix A : Glossary of Terms 50
Appendix B: Technologies Used 50
References 51
Declaration of Authenticity 51
Peer Evaluation 53
Self-Reflection Report 58

Requirement Analysis

1. Introduction

1.1. Purpose

This Requirement Analysis Report documents and analyses the system requirements for the Emergency SOS Services Project (E.SOS App), developed on behalf of POWA (People Opposing Women Abuse). POWA is a South African non-profit organization established in 1979 that provides free counselling, legal services, shelter, advocacy, and public awareness programs for survivors of gender-based violence and domestic abuse (POWA, 2026).

The purpose of this document is to identify, analyze, and formally specify the functional and non-functional requirements, user interactions, system processes, and design features necessary for the successful development and deployment of the E.SOS App. The report serves as the primary reference for the development team, academic supervisor, and client stakeholders throughout the project lifecycle (Sommerville, 2016).

The E.SOS App is a mobile and web-based emergency response platform designed to provide immediate assistance and support services to survivors of gender-based violence, domestic abuse, harassment, and emotional distress. The system aims to improve emergency response times and ensure that vulnerable individuals can access help quickly, safely, and securely during crisis situations. According to Sommerville (2016), requirement analysis is essential for identifying user needs and ensuring that software systems meet stakeholder expectations effectively.

The proposed solution consists of a mobile application for survivors and a web-based dashboard for POWA staff and emergency responders. The mobile application enables users to trigger emergency alerts through a one-tap SOS button, silently request help, share real-time GPS locations, communicate with support personnel, and securely report incidents. The web dashboard allows POWA operators to monitor active alerts, manage incidents, communicate with survivors, and coordinate emergency support services efficiently.

The report further defines the system requirements, user interactions, operational processes, and functional capabilities required for the successful implementation of the E.SOS App. The application aligns with POWA's mission of empowering women and creating safer communities through technology-driven crisis response and long-term support services. Additionally, the system is designed with strong emphasis on accessibility, usability, reliability, and data security to ensure that users with different abilities and technical skills can interact with the platform effectively and safely (Pressman & Maxim, 2020).

1.2. Scope

The E.SOS App is a dual-platform emergency support system comprising a mobile application (Android and iOS) and a web-based operator dashboard. The system is designed to empower survivors of gender-based violence, domestic abuse, harassment, and emotional distress to access immediate emergency assistance. Core capabilities include:

- User registration and secure account management
- One-tap SOS emergency alert activation
- Silent alert functionality for high-risk situations
- Real-time GPS location sharing with emergency contacts and POWA operators
- Secure messaging between survivors and support staff
- Incident reporting with evidence upload functionality
- Offline SMS fallback for low-connectivity environments
- A web dashboard enabling POWA staff to monitor, manage, and respond to alerts

The scope excludes direct integration with South African Police Service (SAPS) automated dispatch systems in the initial release, which is deferred to a future phase subject to partnership agreement (POWA, 2026).

1.3. Overview

This report provides an overview of the Emergency SOS Services Project (E.SOS App) developed for POWA (People Opposing Women Abuse). The system is designed to assist survivors of gender-based violence, domestic abuse, harassment, and emotional distress through emergency response and support services.

The report is divided into two main sections. Part One focuses on the Requirement Analysis, including the solution domain, system environment, functional and non-functional requirements, UML use case diagrams, logical system models, and class diagrams. Part Two presents the System Design Document, which includes the logical architecture, user interface design, database design, entity relationship diagrams, system reports, and supporting appendices.

The report also discusses the technologies and platforms used in the system, such as mobile applications, web dashboards, GPS tracking, cloud databases, and emergency communication services. Harvard Anglia referencing style is applied throughout the document.

2. Solution Domain

2.1. System Environment

The E.SOS App operates within the emergency response and social support domain. The system combines mobile technology, cloud computing, GPS tracking, secure communication, and emergency notification services to provide real-time support for survivors of abuse, violence, harassment, and emotional distress.

The application supports both emergency crisis intervention and long-term survivor assistance through SOS alerts, incident reporting, counselling support, and communication services. The system is designed to improve response times, enhance user safety, and provide reliable access to emergency support services.

The E.SOS App includes a mobile platform for survivors and a web-based dashboard for POWA staff and emergency responders. The system integrates GPS services, cloud databases, and SMS notifications to ensure efficient emergency response management.

Component	Description
Mobile Application	Android and iOS applications for survivors to access emergency assistance and support services at any time.
Web Dashboard	A browser-based system enables POWA staff and operators to monitor alerts, manage incidents, and coordinate emergency responses in real time.
GPS Tracking Services	Real-time location tracking using Google Maps Platform APIs to improve emergency response accuracy and reduce response times.
SMS Gateway (Twilio)	Enables emergency notifications and SOS fallback alerts to be transmitted via SMS when internet connectivity is unavailable.
Secure Cloud Database	Firebase / AWS-hosted database that stores user profiles, incident reports, emergency contact details, GPS logs, and communication records with AES-256 encryption at rest.
Cloud Infrastructure	AWS Elastic Beanstalk / Azure App Services for scalable hosting, automated failover, and continuous availability of the platform.
RESTful API Layer	Node.js backend exposing RESTful endpoints for mobile and web clients, handling authentication, alert processing, messaging, and reporting.
Emergency Services Directory	Curated directory of POWA shelters, legal aid services, counselling contacts, and national emergency hotlines

	(SAPS 10111, Ambulance 10177, GBV Command Centre 0800-428-428).

2.2. User Characteristics

The E.SOS App is designed to serve multiple categories of users, each with different technical abilities, responsibilities, and interaction needs. Understanding these user characteristics is essential to ensure that the system is accessible, user-friendly, and effective during emergency situations where quick and accurate response is critical. The system is therefore structured to accommodate both non-technical users who require simple interactions and highly technical users who manage and maintain the platform.

User Type	Technical Profile	Interaction Characteristics
Survivors / Users	Smartphone users (Android/iOS) with varying levels of digital literacy	Require a simple, fast, and intuitive mobile interface. Often under high stress during emergencies, the system must support quick SOS activation, minimal navigation, and clear instructions. Primary access is through the mobile application for emergency alerts and support services.
POWA Operators	Moderately computer-literate staff using desktop or laptop systems	Responsible for monitoring incoming alerts, managing incidents, communicating with survivors, and coordinating emergency responses through the web dashboard. Require real-time updates and an easy-to-use monitoring interface.
System Administrators	Highly technically proficient IT personnel	Manage system configuration, user accounts, security settings, database maintenance, and system updates. Require full administrative access and control over backend infrastructure.
Emergency Responders	SAPS, EMS, and field response teams with mobile or desktop access	Receive real-time, location-based alerts and incident details to enable rapid physical response. Require accurate GPS information and clear incident summaries to support urgent decision-making.

2.3. Assumptions and Dependencies

The design and implementation of the E.SOS App are based on several key assumptions and external dependencies that must remain valid for the system to function effectively and reliably in real-world emergency scenarios.

- Users are assumed to own smartphones running Android 10 or higher, or iOS 14 or higher, with GPS functionality enabled to support accurate location tracking.
- A stable internet connection (3G, 4G, 5G, or Wi-Fi) is required for full system functionality, while SMS-based fallback services will operate using any available cellular network signal.
- Third-party service providers such as Google Maps Platform, Twilio, and Firebase are assumed to remain available, stable, and accessible through valid service agreements throughout the system lifecycle.
- POWA will provide trained operators with access to the web dashboard and support user onboarding and system training prior to deployment.
- The system assumes compliance with the Protection of Personal Information Act (POPIA), 2013, ensuring that all personal and sensitive user data is securely processed and protected under South African legal requirements (South African Government, 2013).
- Cloud infrastructure providers (such as AWS or Azure) are assumed to maintain service availability with a minimum uptime service level agreement (SLA) of approximately 99.5% to ensure system reliability during emergencies.

3. Functional Requirements

3.1. Functional Requirements Master List

The following table lists the core functional requirements for the E.SOS App.

Req. ID	Requirement Name	Description
FR-01	**User Registration and Login**	Users must be able to create secure accounts and authenticate using credentials. Supports password reset via verified email or SMS OTP.
FR-02	**One-Tap SOS Alert**	Users must be able to trigger an immediate emergency alert using a single large, accessible button on the home screen.
FR-03	**Silent Alert Functionality**	Users must be able to request emergency assistance covertly, without producing audible or visual output that may endanger them.
FR-04	**Real-Time GPS Location Sharing**	The system must capture and transmit the user's live GPS coordinates to pre-registered emergency contacts and POWA operators upon alert activation.
FR-05	**Automated Emergency Notifications**	The system must deliver instant notifications via push notification, SMS (Twilio), and email upon SOS activation.
FR-06	**Incident Reporting**	Users must be able to create structured incident reports and attach supporting evidence (photos, voice notes, written descriptions).
FR-07	**Secure Messaging**	The system must provide end-to-end encrypted messaging between survivors and authorized POWA counsellors or support staff.
FR-08	**Operator Dashboard**	POWA operators must access a web-based dashboard to monitor incoming alerts, view GPS locations, manage active cases, and update incident statuses.
FR-09	**Emergency Hotline Directory**	The application must provide a categorized, searchable directory of emergency contact numbers including SAPS, ambulance

		services, GBV support lines, and POWA shelters.
FR-10	**Offline SOS (SMS Fallback)**	When internet connectivity is unavailable, the system must automatically fall back to SMS-based SOS notification using Twilio's messaging API.
FR-11	**User Profile Management**	Users must be able to manage their personal profile including emergency contacts, medical information, and notification preferences.
FR-12	**Incident History and Case Tracking**	The system must store and display a chronological history of all alerts and incidents for both users and operators.
FR-13	**Multi-Language Support**	The application must support at least English and Zulu to improve accessibility across South African linguistic communities.
FR-14	**Awareness and Support Resources**	The application must provide educational content on abuse prevention, survivor rights, and recovery pathways curated by POWA.

3.2. Non-Functional Requirements

Category	Requirement
Performance	SOS alerts must be processed and delivered within three seconds of activation. The system must support at least 500 concurrent users without performance degradation.
Security	All data at rest must be encrypted with AES-256. All data in transit must use TLS 1.3. Authentication must use JWT-based tokens. The system must comply with OWASP Top 10 security guidelines.
Availability	The system must maintain 99.5% uptime, supported by automated failover and cloud-based redundancy. Scheduled maintenance windows must not exceed two hours per month.
Accessibility	The mobile application must comply with WCAG 2.1 Level AA accessibility standards, supporting screen readers, adjustable font sizes, and high-contrast display modes.
POPIA Compliance	All personal information collected must comply with the Protection of Personal Information Act, 2013 (POPIA). Data minimization, explicit consent management, and user data deletion rights must be implemented (South African Government, 2013).
Scalability	The cloud architecture must scale horizontally to accommodate growth in POWA's user base, using containerised deployments (Docker) on AWS or Azure.
Reliability	The SMS fallback mechanism must activate automatically and reliably whenever internet connectivity is unavailable. System failures must trigger automated alerts to the system administrator.

3.3. Detailed Functional Requirement Specifications

FR-02: One-Tap SOS Alert

Description	A single prominent button on the application home screen enables users to dispatch an emergency SOS alert immediately.
System Input	Single tap on the SOS button by the authenticated user.

System Processing	The system captures the current GPS coordinates, identifies pre-registered emergency contacts, packages the alert payload, and dispatches it via push notification, SMS, and email channels.
System Output	Emergency notifications delivered to all registered contacts and POWA operator dashboard within three seconds. An alert confirmation is displayed to the user.
Error Handling	If GPS is unavailable, the last known location is transmitted with a 'location approximate' flag. If network is unavailable, SMS fallback activates automatically.
Constraints	The user must be authenticated. The device must have at least one registered emergency contact. The SOS button must remain accessible from the home screen without navigation.

FR-03: Silent Alert Functionality

Description	Allows a user to request emergency assistance without triggering any sound, vibration, or visible on-screen indication that may expose them to the abuser.
System Input	Activation via a disguised interface element (e.g., volume button sequence or hidden menu) configured during initial setup.
System Processing	Identical alert processing to FR-02, with all device audio and visual feedback suppressed. GPS coordinates captured silently.
System Output	Silent dispatch of notifications to POWA operators and emergency contacts. No on-screen confirmation.
Constraints	The silent trigger mechanism must be configured by the user prior to a crisis event. Device accessibility settings must not override silence suppression.

FR-06: Incident Reporting

Description	Users can create structured reports of abuse incidents, attaching photographic, audio, or written evidence for legal and counselling purposes.
System Input	User-completed form fields (incident type, date, location, description) plus optional media attachments (photos, voice notes).
System Processing	The system validates and sanitises form inputs, encrypts media attachments, stores the report in the secure cloud database, and assigns a unique incident reference number.
System Output	Confirmation message with incident reference number. Report accessible to the user in their incident history and to authorised POWA operators.
Data Handling	All media files are encrypted at rest (AES-256). Report data is retained for a minimum of five years in compliance with POPIA data retention provisions.

4. UML Use Case Diagram Background

4.1. System Actors

The E.SOS App system consists of multiple actors who interact with the system to perform specific functions. These actors represent both primary users of the system and support external entities involved in emergency response and system management.

Actor	Type	Role Description
Survivor / User	Primary	A person experiencing abuse or distress who uses the mobile application to trigger

		emergency alerts, submit incident reports, and access support services.
POWA Operator	Primary	Staff member responsible for monitoring alerts, managing cases, communicating with survivors, and coordinating emergency responses through the web dashboard.
Emergency Responder	Secondary	SAPS or emergency medical personnel who receive location-based alerts and respond physically to incidents.
System Administrator	Secondary	Technical user responsible for managing system configuration, user accounts, security settings, and overall system maintenance.

4.2. Use Cases

The following use cases describe the main functional interactions between system actors and the E.SOS App.

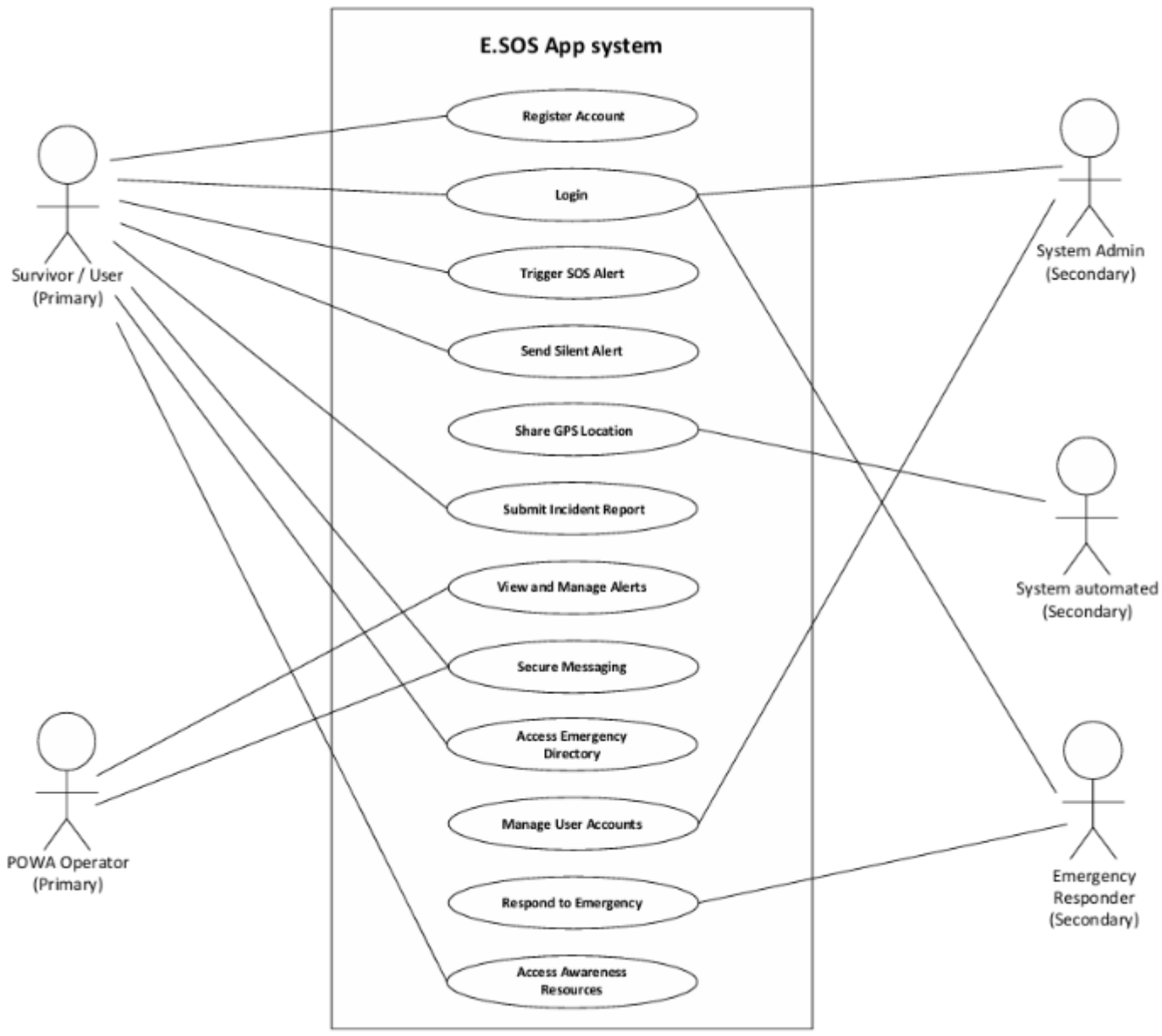
UC ID	Use Case Name	Primary Actor	Description
UC-01	**Register Account**	Survivor / User	User creates an account by providing personal details, setting credentials, and adding emergency contacts.
UC-02	**Login**	All Users	Users authenticate into the system using secure credentials. A JWT token is issued upon successful login.
UC-03	**Trigger SOS Alert**	Survivor / User	User activates the emergency SOS button, sending an alert with real-time GPS location to operators and emergency contacts.
UC-04	**Send Silent Alert**	Survivor / User	User discreetly triggers an SOS alert without any visible or audible notification on the device.
UC-05	**Share GPS Location**	System (automated)	The system automatically captures and transmits real-time GPS coordinates when an alert is triggered.
UC-06	**Submit Incident Report**	Survivor / User	User submits a structured report describing the incident, with optional media attachments.
UC-07	**View and Manage Alerts**	POWA Operator	Operator monitors incoming alerts, updates case status, and assigns incidents for response.
UC-08	**Secure Messaging**	Survivor / Operator	Enables encrypted communication between survivors and operators for ongoing support.
UC-09	**Access Emergency Directory**	Survivor / User	Users access a directory of emergency contacts, shelters, counselling services, and hotlines.

UC-10	**Manage User Accounts**	System Administrator	Administrator creates, updates, or deactivates user and operator accounts.
UC-11	**Respond to Emergency**	Emergency Responder	Emergency responders receive alert notifications with GPS data and respond to incidents on-site.
UC-12	**Access Awareness Resources**	Survivor / User	User accesses educational content on abuse prevention, rights, and support services.

4.3. Use Case Relationships

The following relationships define how use cases interact within the E.SOS App system:

- UC-03 (Trigger SOS Alert) <<include>> UC-05 (Share GPS Location) – GPS capture is mandatory on alert activation.
- UC-04 (Send Silent Alert) <<extend>> UC-03 (Trigger SOS Alert) – Silent alert is a specialised variant of the standard SOS alert.
- UC-03 and UC-04 <<include>> automated notification dispatch to all registered emergency contacts.
- UC-06 (Submit Incident Report) <<extend>> UC-03 – A user may optionally create an incident report following an SOS activation.
- UC-07 (View and Manage Alerts) <<include>> UC-11 (Respond to Emergency) – Operators may escalate alerts to emergency responders.
- UC-01 (Register Account) is a precondition for all user-facing use cases.
- UC-02 (Login) <<include>> all authenticated use cases.



5. Logical System Model

The logical system model describes the flow of information through the E.SOS App from input to processing and output. It provides an abstract representation of how data moves through the system, forming the foundation for database design and system architecture decisions (Dennis, Wixom and Roth, 2012).

5.1. System Inputs

The following inputs are captured and processed by the system:

Input Type	Description
User Registration Data	Includes full name, contact number, email address, password, profile photo, emergency contact details, and optional medical information.
SOS Alert Request	Emergency trigger initiated through a button press or silent activation mechanism.
GPS Coordinates	Real-time latitude and longitude data captured from the device's GPS module at the time of alert activation.
Incident Report Data	Includes incident type, description, date and time, location, and optional media such as images or voice recordings.
Messaging Content	Encrypted messages exchanged between users and POWA operators via the secure messaging system.
Authentication Credentials	Login details such as email/username and password, including OTP codes for password recovery.

5.2. System Processes

The system processes transform inputs into meaningful outputs to support emergency response and case management.

Process	Description
User Authentication	Verifies user credentials against encrypted database records and generates a JWT token upon successful login.
Alert Processing	Handles SOS triggers, retrieves emergency contacts, attaches GPS data, and sends notifications through multiple channels.
GPS Processing	Captures device location, converts coordinates into a map link, and transmits location data to relevant recipients.
Notification Dispatch	Sends alerts via push notifications, SMS (Twilio), and email depending on network availability and user settings.
Incident Storage	Encrypts and stores incident reports and attachments in secure cloud storage with unique reference identifiers.
Message Encryption	Ensures all in-app communications are encrypted end-to-end before transmission and decrypted only by authorised users.
Dashboard Processing	Processes incoming alerts and displays real-time updates, including location mapping and case status, on the operator dashboard.

5.3. System Outputs

Output Type	Description
Emergency Notifications	Real-time alerts sent via push notifications, SMS, and email to emergency contacts and operators.
Real-Time Dashboard Alerts	Live incident updates displayed on the POWA dashboard, including user details and location data.
GPS Location Transmission	Google Maps-based location links shared with responders and emergency contacts.
Incident Reports	Structured, encrypted records of reported incidents accessible to authorised users and operators.

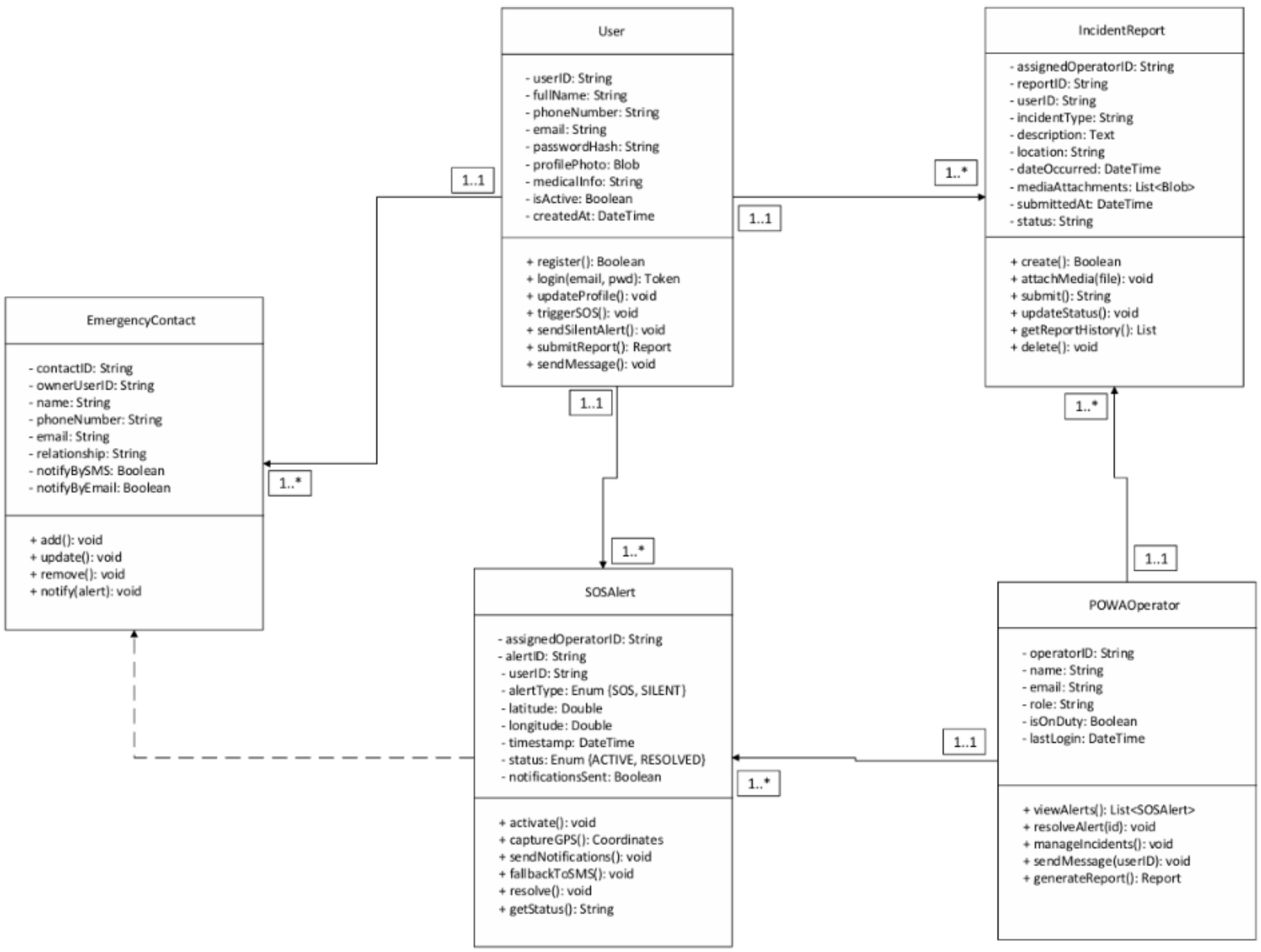
System Reports	Analytical reports generated for management, including response times, alert frequency, and trend analysis.
Audit Logs	Secure logs of system activity for monitoring, security tracking, and POPIA compliance.

5.4. Data Storage

The E.SOS App uses a combination of cloud-based relational and NoSQL databases to securely store and manage system data:

- **User Profiles:** Encrypted records containing personal details, emergency contacts, and authentication data.
- **Alert Records:** Complete history of SOS activations, including timestamps, GPS coordinates, and response status.
- **Incident Reports:** Structured incident data with associated media stored in encrypted cloud storage.
- **Communication Logs:** Secure and encrypted message histories between survivors and operators.
- **Audit Logs:** System activity records used for security monitoring and legal compliance.
- **GPS History:** Time-stamped location data linked to emergency events, stored according to POPIA retention requirements.
- **Operator Activity Logs:** Records of all actions performed by operators for accountability and system auditing.

6. Class Diagrams



Class Relationships

Relationship	Type	Description

User → SOSAlert	Composition (1 to many)	A User may trigger many SOSAlerts over time. An alert cannot exist without an owning User.
User → IncidentReport	Composition (1 to many)	A User may submit many IncidentReports. Reports are owned by the submitting User.
User → EmergencyContact	Aggregation (1 to many)	A User registers one or more EmergencyContacts. Contacts can exist independently of alert events.
POWAOperator → SOSAlert	Association (1 to many)	One POWAOperator manages multiple SOSAlerts, but each SOSAlert is assigned to only one POWAOperator at a time.
POWAOperator → IncidentReport	Association (1 to many)	One POWAOperator reviews and manages multiple IncidentReports, but each IncidentReport is handled by only one POWAOperator.
SOSAlert → EmergencyContact	Dependency	SOSAlert triggers the notify() method on all EmergencyContacts registered to the User.

System Design Document

7. Introduction

The System Design Document (SDD) translates the functional and non-functional requirements identified in Part One into a detailed technical design for the E.SOS App. The document explains how the system will be developed, how different system components will interact, and how data will be processed, stored, and secured. The SDD serves as the main technical reference for developers, system administrators, and project stakeholders during the implementation phase of the project lifecycle (Pressman, 2014).

The E.SOS App is designed using a three-tier client-server architecture consisting of a presentation layer, an application layer, and a cloud-hosted data layer. The presentation layer includes the mobile application for survivors and the web dashboard for POWA staff and emergency responders. The application layer uses a Node.js backend to process requests, manage authentication, handle SOS alerts, and support communication services. The data layer securely stores user information, incident reports, GPS data, and communication records within a cloud-hosted database environment.

The system design prioritises security, scalability, reliability, and usability to ensure that emergency support services remain continuously available during crisis situations. In addition, the platform is designed to comply with the Protection of Personal Information Act (POPIA) by implementing secure authentication, encrypted communication, controlled access management, and protected data storage mechanisms.

8. Logical Architecture Design – High-Level Architectural Design

8.1. Architectural Overview

The E.SOS App uses a three-tier Model-View-Controller (MVC) architecture hosted on cloud infrastructure to ensure scalability, reliability, security, and high system availability (Pressman, 2014). The architecture separates the presentation, application, and data management functions to improve maintainability and performance.

Tier	Components	Description
Presentation Layer (View)	React Native Mobile App, React.js Web Dashboard	Provides user-facing interfaces for survivors, POWA operators, and administrators. The mobile application supports Android and iOS devices, while

		the web dashboard enables browser-based monitoring and management.
Application Layer (Controller)	Node.js / Express.js RESTful API, JWT Authentication, Firebase Cloud Messaging, Twilio SMS	Handles business logic, authentication, SOS alert processing, notification management, secure messaging, and communication with third-party services.
Data Layer (Model)	Firebase Firestore / PostgreSQL, AWS S3 (media), Redis (cache)	Manages secure storage of user data, alerts, incident reports, GPS records, communication logs, and media attachments. Redis is used to improve performance through caching.

8.2. Technology Stack

Component	Technology	Justification
Mobile App	React Native / Flutter	Supports both Android and iOS using a single codebase, reducing development and maintenance costs.
Web Dashboard	React.js	Provides a responsive and real-time interface for monitoring alerts and managing incidents.
Backend API	Node.js + Express.js	Supports high-concurrency real-time processing and efficient API communication.
Primary Database	Firebase Firestore	Real-time NoSQL database suitable for live alert updates and offline synchronisation.
Relational Database	PostgreSQL	Stores structured records such as incident reports, audit logs, and user profiles requiring relational integrity.
Media Storage	AWS S3	Secure and scalable cloud storage for images, voice notes, and other uploaded media files.
Push Notifications	Firebase Cloud Messaging	Delivers real-time notifications to Android and iOS devices.
SMS Gateway	Twilio	Sends SMS notifications when internet connectivity is unavailable.
Location Services	Google Maps Platform	Provides GPS tracking, geolocation services, and shareable map links.
Authentication	JWT + Firebase Auth	Enables secure user authentication and token-based access control.
Containerisation	Docker + Kubernetes	Supports scalable deployment, automated failover, and efficient resource management.

8.3. Security Architecture

Security is integrated into all layers of the E.SOS App architecture to ensure compliance with OWASP security principles and the Protection of Personal Information Act (POPIA) (South African Government, 2013).

- Transport Layer Security (TLS 1.3) is used to secure all API communications and data transmissions.
- AES-256 encryption is applied to sensitive data stored within databases and cloud storage systems.

- JWT authentication tokens use short expiration periods and refresh token rotation to reduce session hijacking risks.
- Role-Based Access Control (RBAC) restricts system access according to user roles and permissions.
- Input validation, sanitisation, and parameterised queries are implemented to prevent SQL injection and cross-site scripting (XSS) attacks.
- API rate limiting and DDoS protection mechanisms are used to protect the platform from malicious traffic and abuse.
- Regular penetration testing and vulnerability assessments are conducted during the system testing phase to identify and resolve security weaknesses.

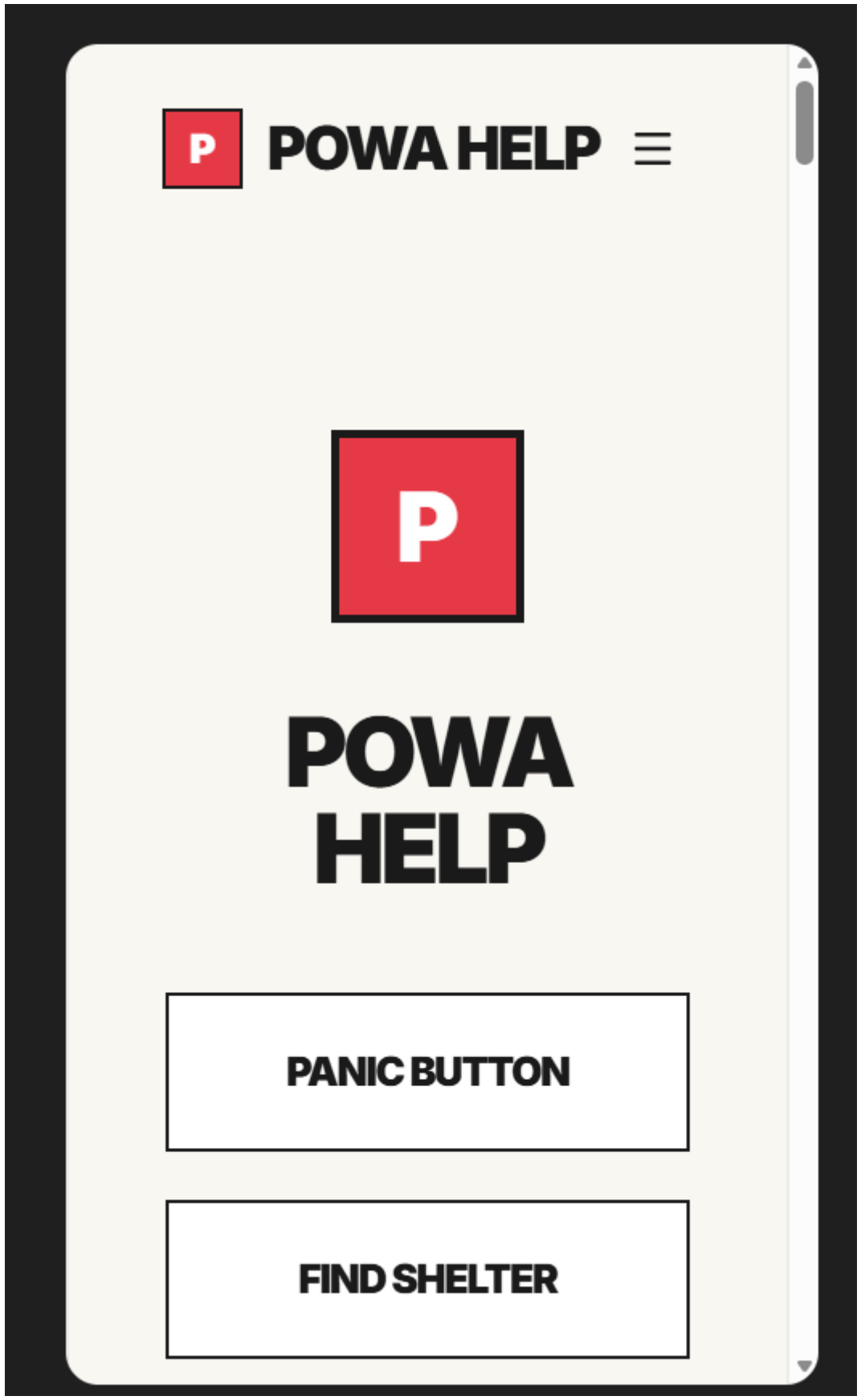
9. Interaction with User – Input Interactions

9.1. Mobile Application – User interaction Flow

SUPPORT FEATURES:

Secure Reporting: A unique form that allows users to safely report issues.

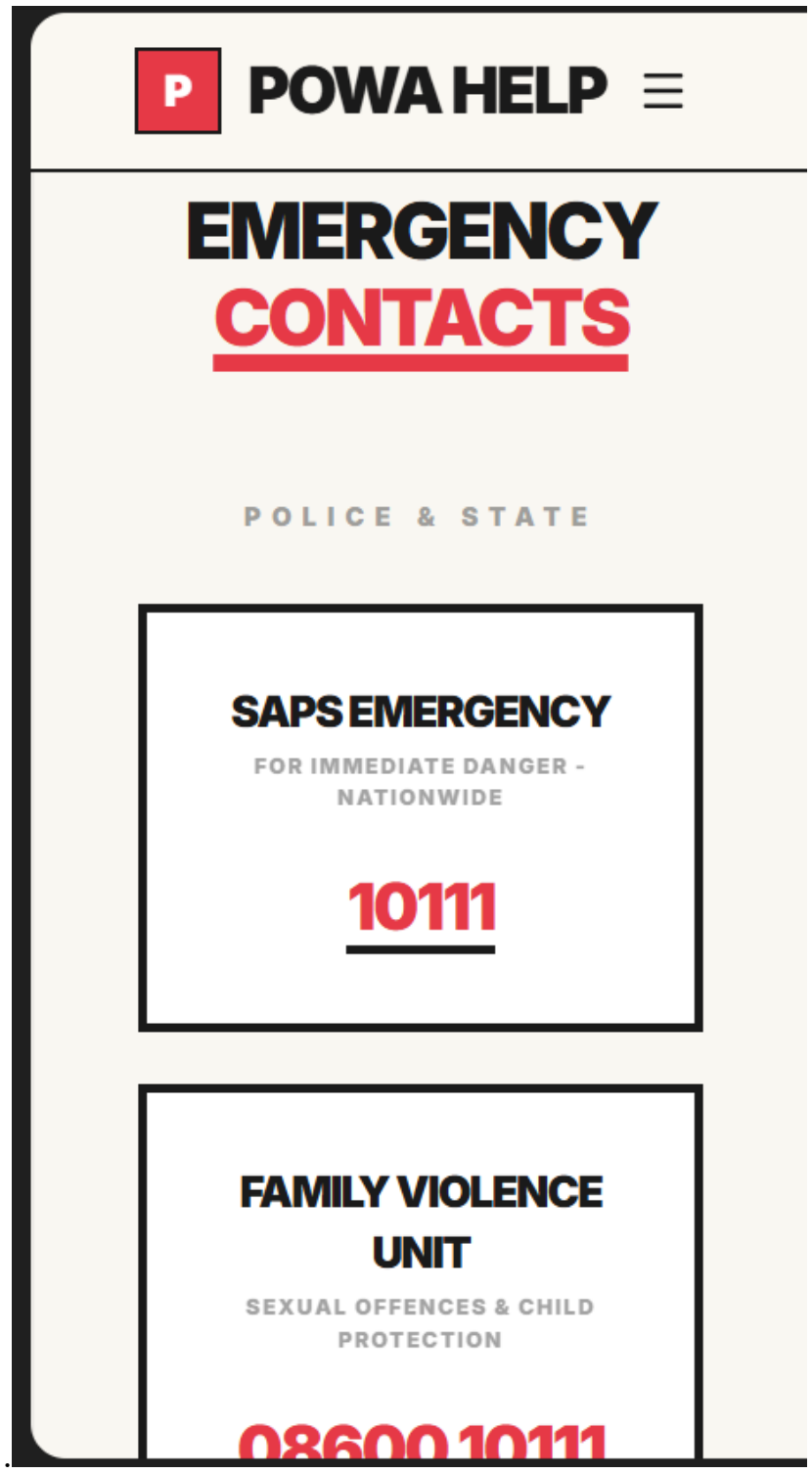
Report Details: The form has a text field for incident details, a dropdown menu to choose the kind of incident (e.g., "Physical Abuse"), and an optional field for the user's name.

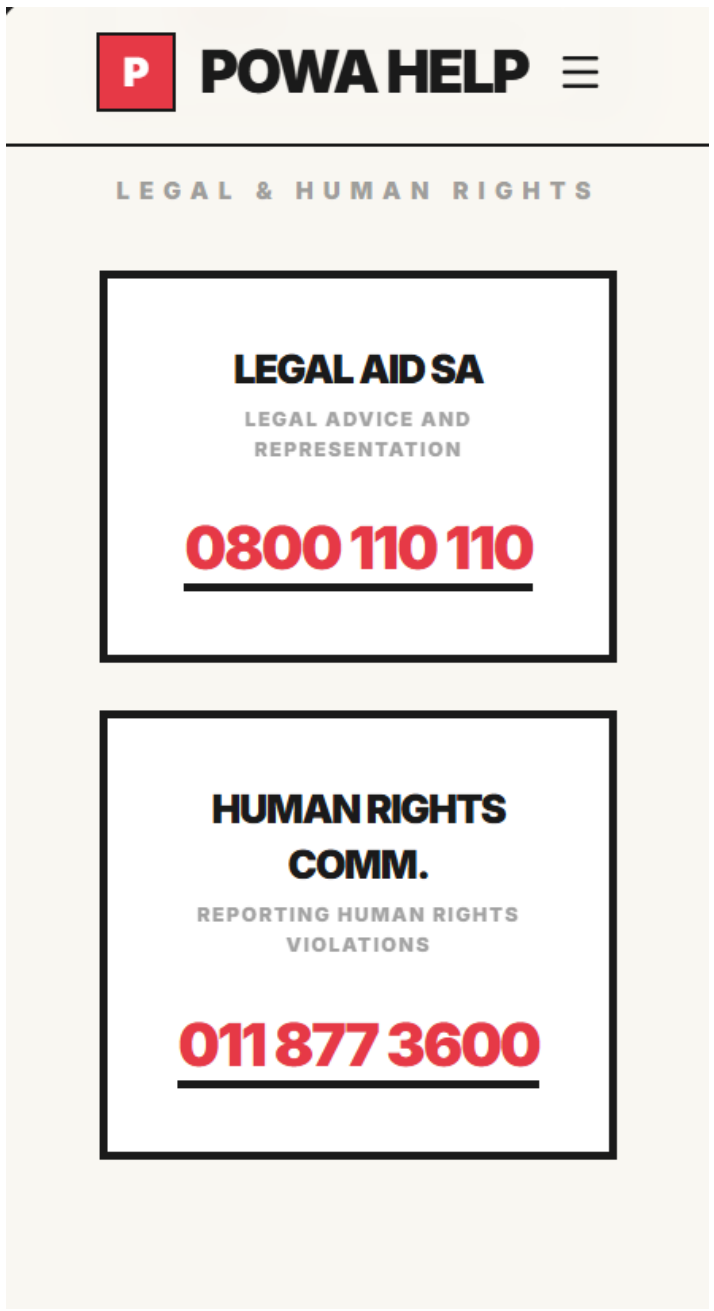


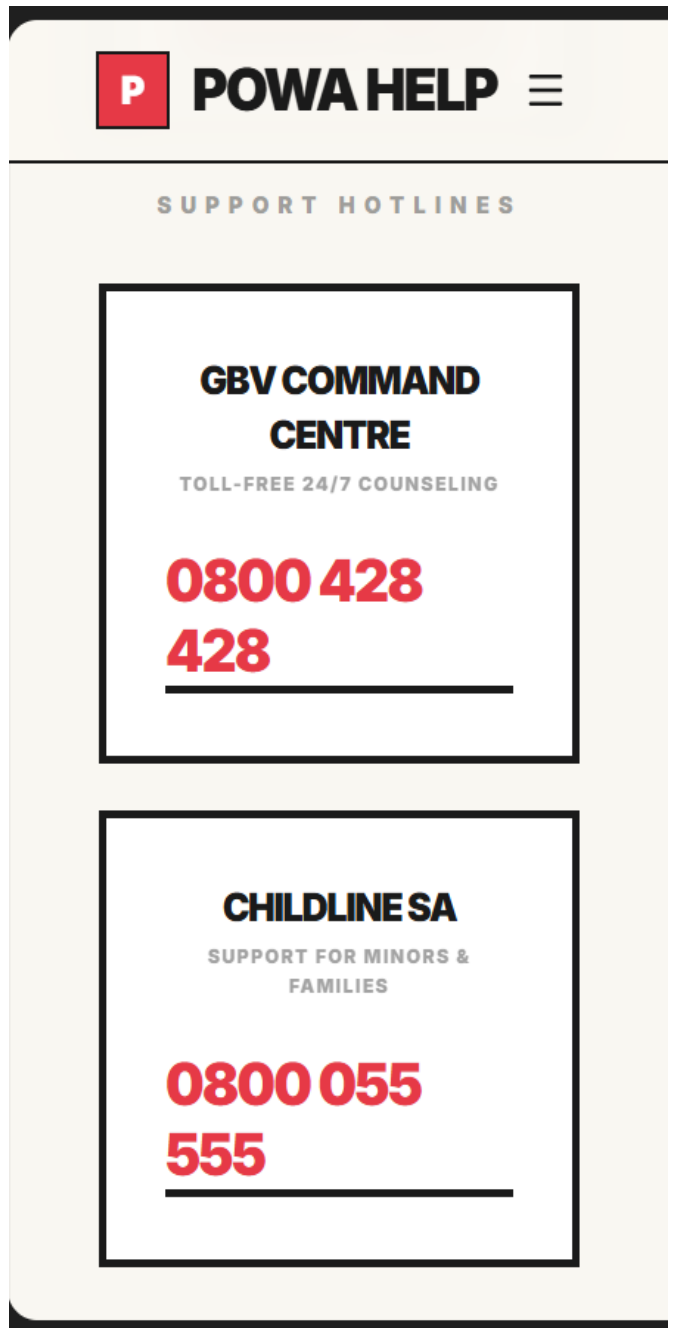
LEGAL AND HUMAN RIGHTS:

Secure Reporting: A unique form that allows users to safely report issues.

Report Details: The form has a text field for incident details, a dropdown menu to choose the kind of incident (e.g., "Physical Abuse"), and an optional field for the user's name


POWA HELP
EMERGENCY
CONTACTS
POLICE & STATE
SAPS EMERGENCY
FOR IMMEDIATE DANGER - NATIONWIDE
10111
FAMILY VIOLENCE UNIT
SEXUAL OFFENCES & CHILD PROTECTION
08600 10111





RESOURCE MANAGMENT:

Key Resource Categories: "Know Your Rights" and "Signs of Abuse" are two important information areas that may be accessed through the platform. Support Services: "Legal Aid" and "Safe Shelters" have their own sections.







RESOURCES





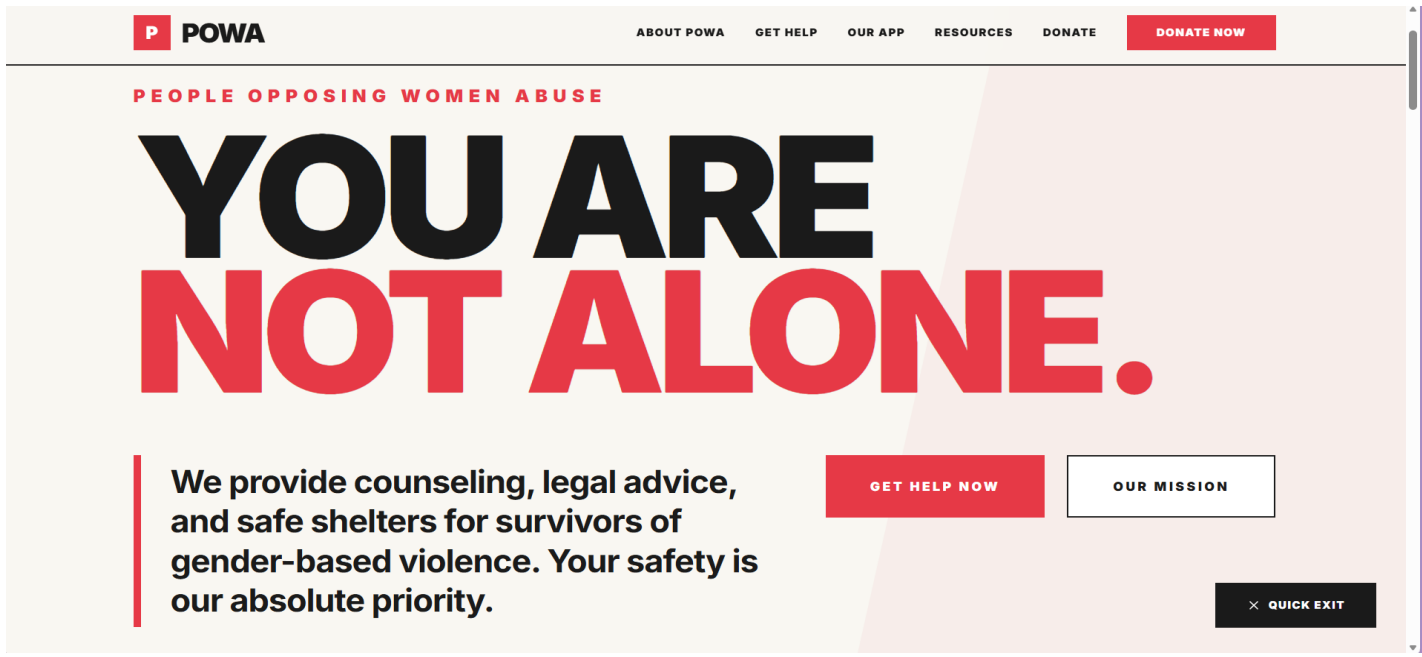




9.2. Web Dashboard – Operator Interaction Flow

HOME AND LANDING PAGE:

This is the main landing page with the encouraging headline, "YOU ARE NOT ALONE." It outlines the organization's primary offerings, which include secure shelters, legal help, and therapy. "Get Help Now" and "Our Mission" buttons, as well as a constant "Quick Exit" safety option, are important interactive components.



ABOUT :

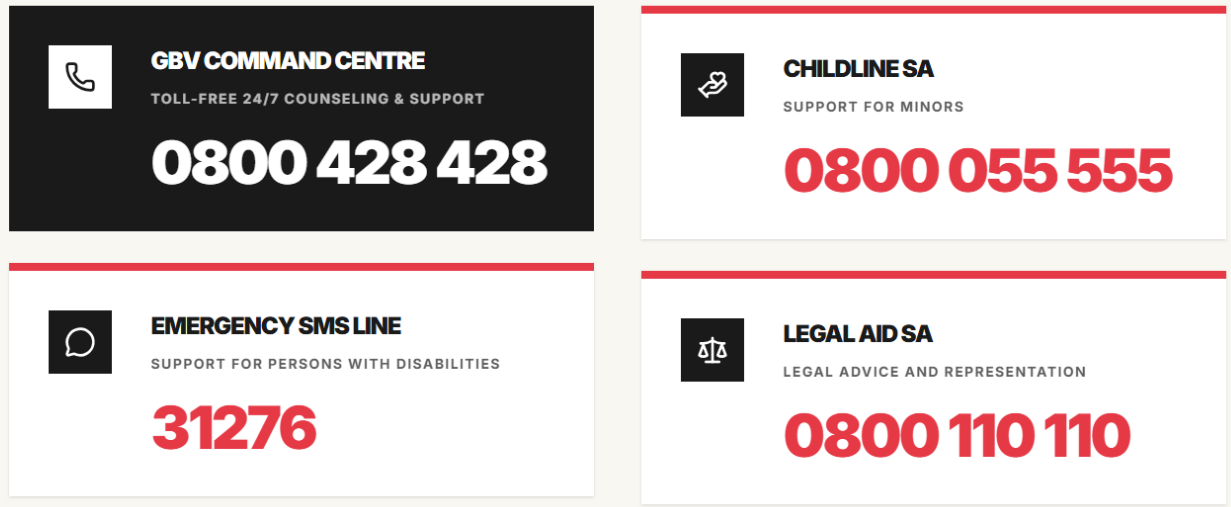
This page, titled **"OVER 40 YEARS OF RESISTANCE,"** details the history of POWA since its establishment in 1979. It features the organization's vision and mission statements and highlights its legacy as the first organization in South Africa to establish a shelter for abused women.



GET HELP:

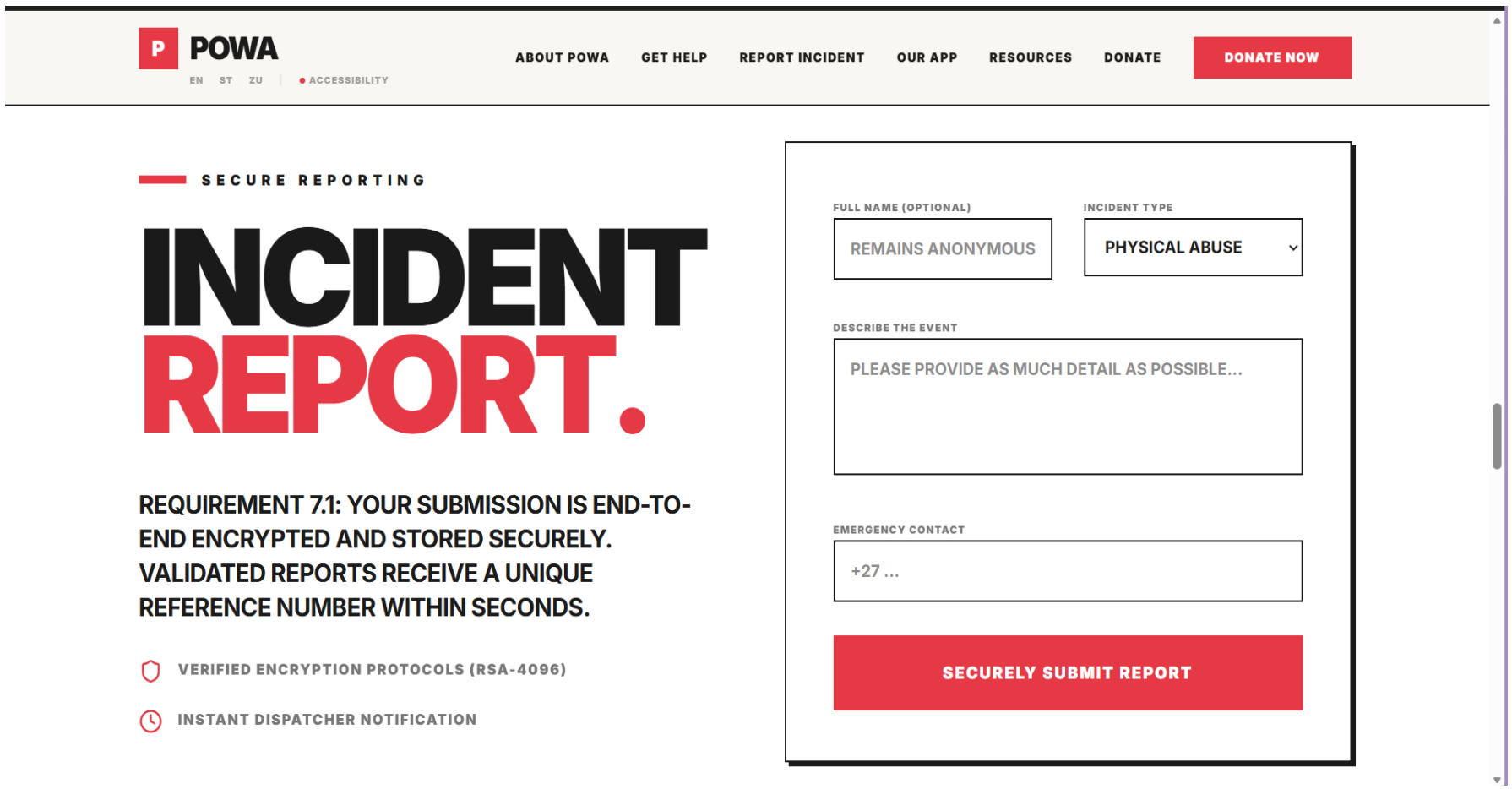
Important emergency numbers, such as the Police Emergency (10111), GBV Command Center, SAPS Family Violence section, and Childline SA, are listed in a neat, prominent layout.





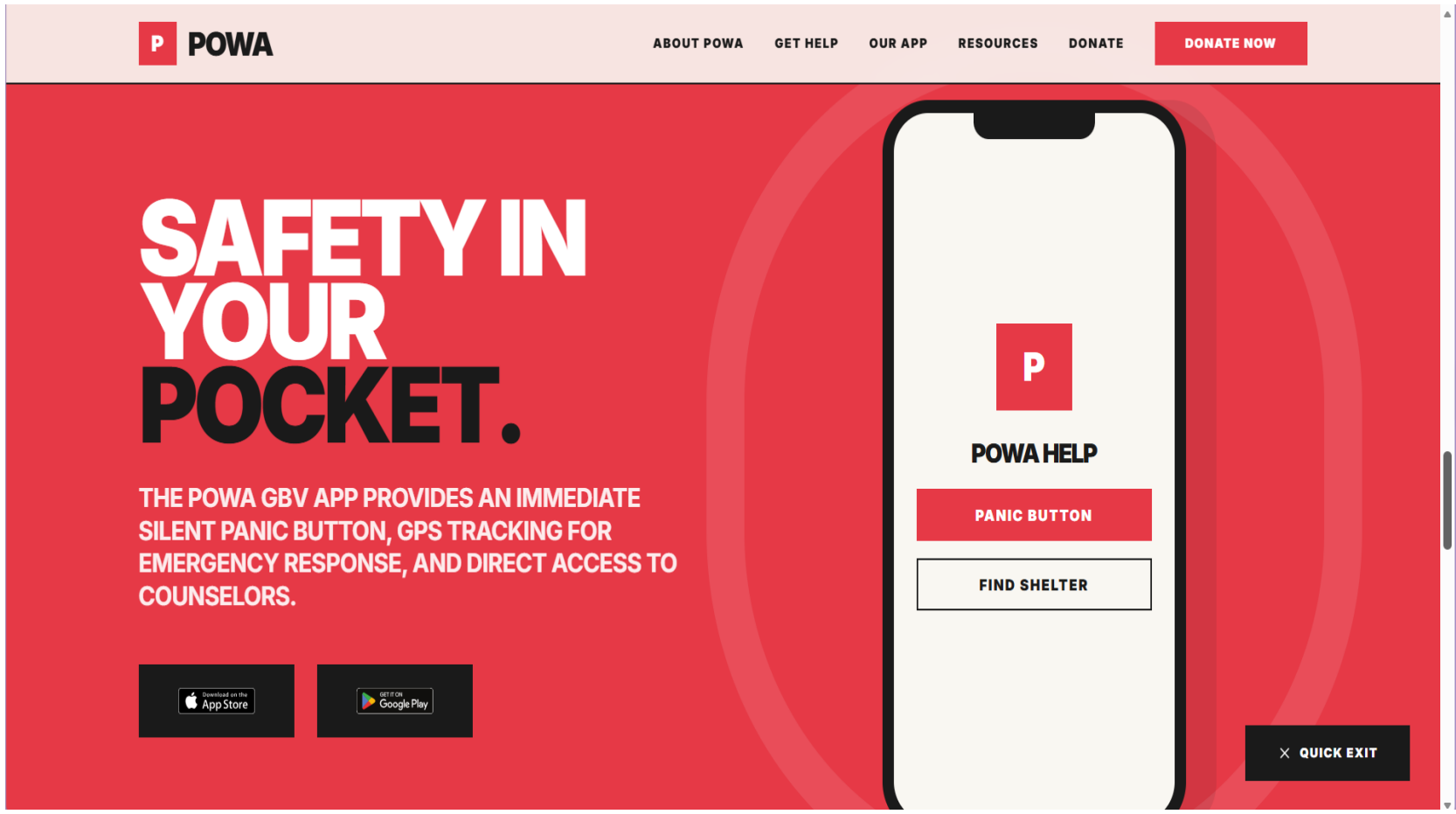
REPORT INCIDENT:

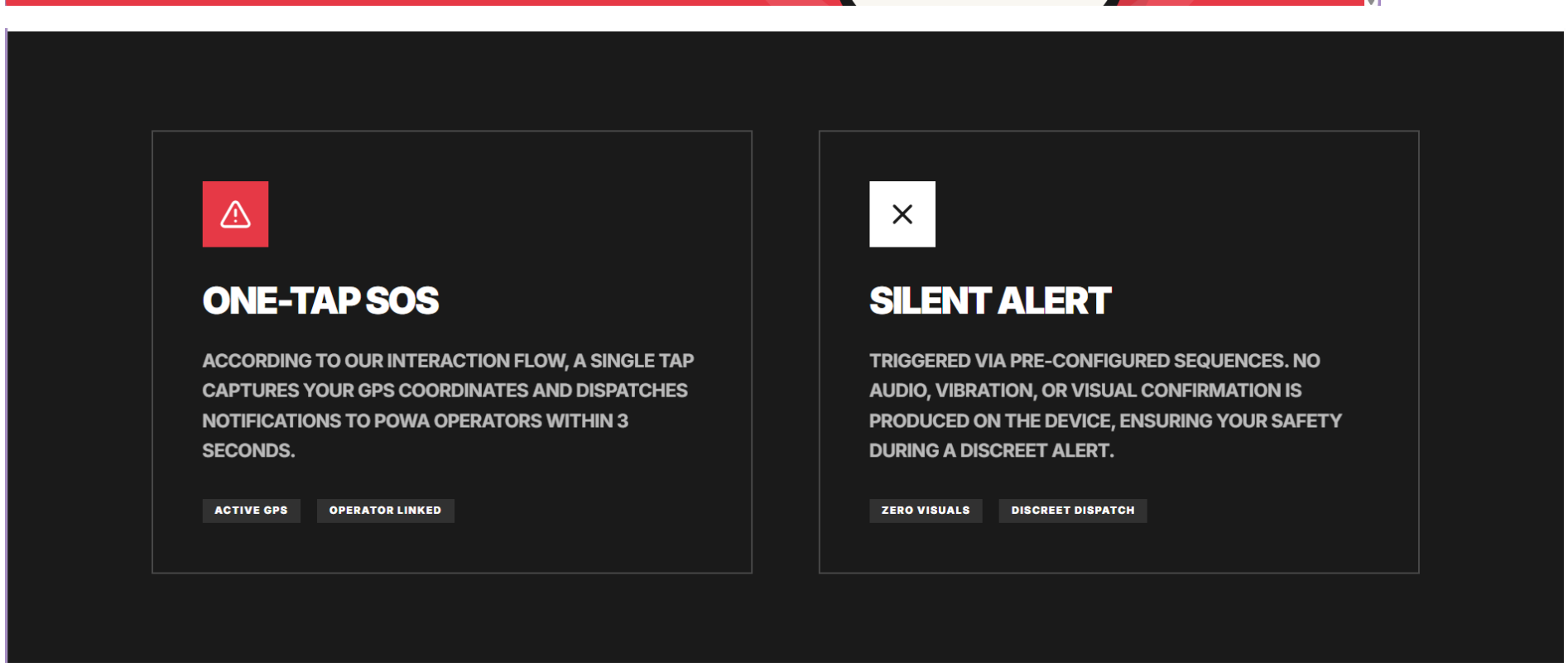
a specific form for submitting a "INCIDENT REPORT." It highlights security by bringing end-to-end data protection and RSA-4096 encryption. The form has fields for incident type, event description, and emergency contact, and permits anonymous reporting.



OUR APP:

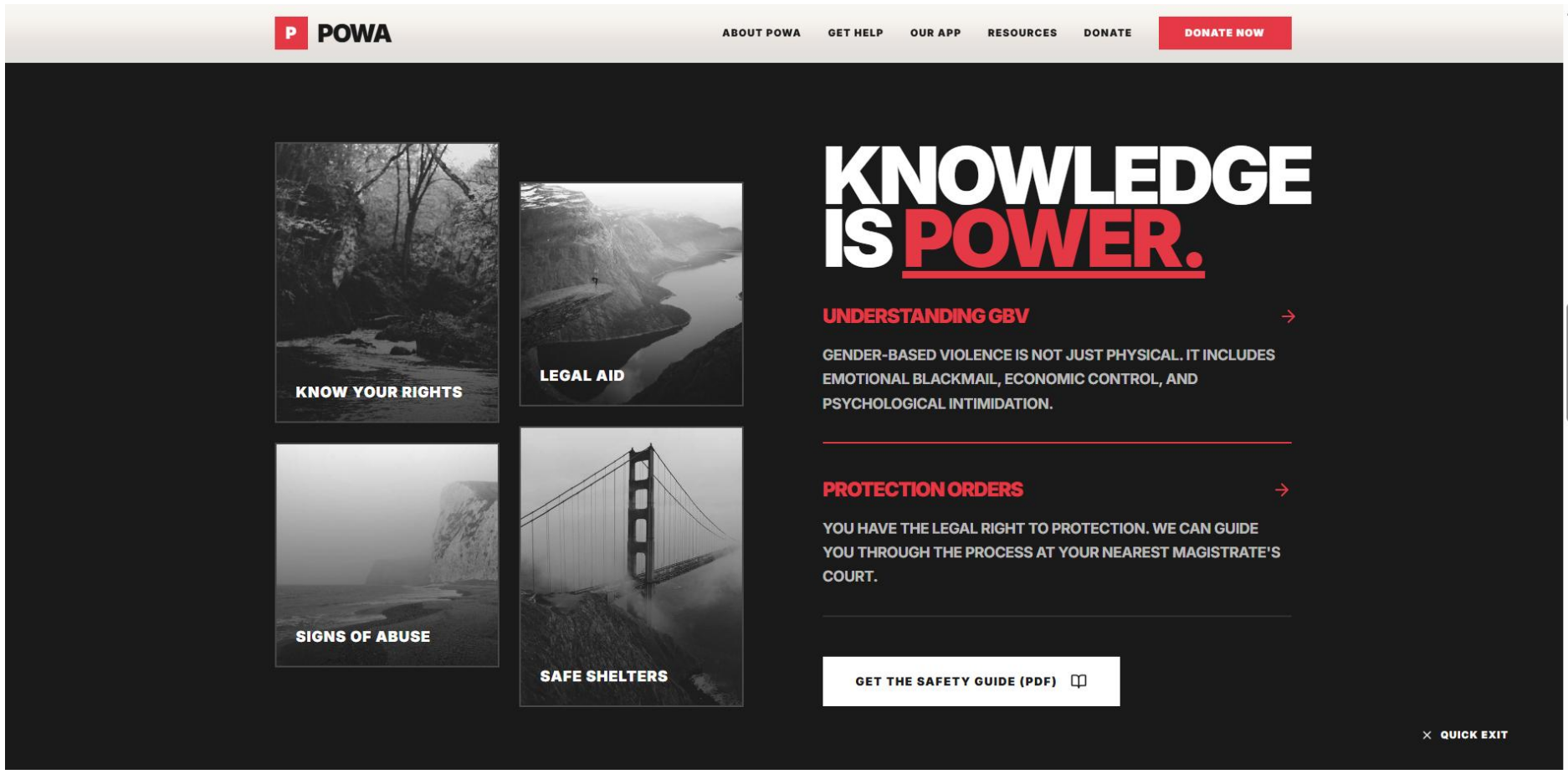
A thorough examination of the app's security features. One-Tap SOS: Provides operators with GPS coordinates in three seconds. Silent Alert: A covert trigger that guarantees the user's safety in an emergency by not producing any visible, auditory, or vibrational confirmation on the gadget.





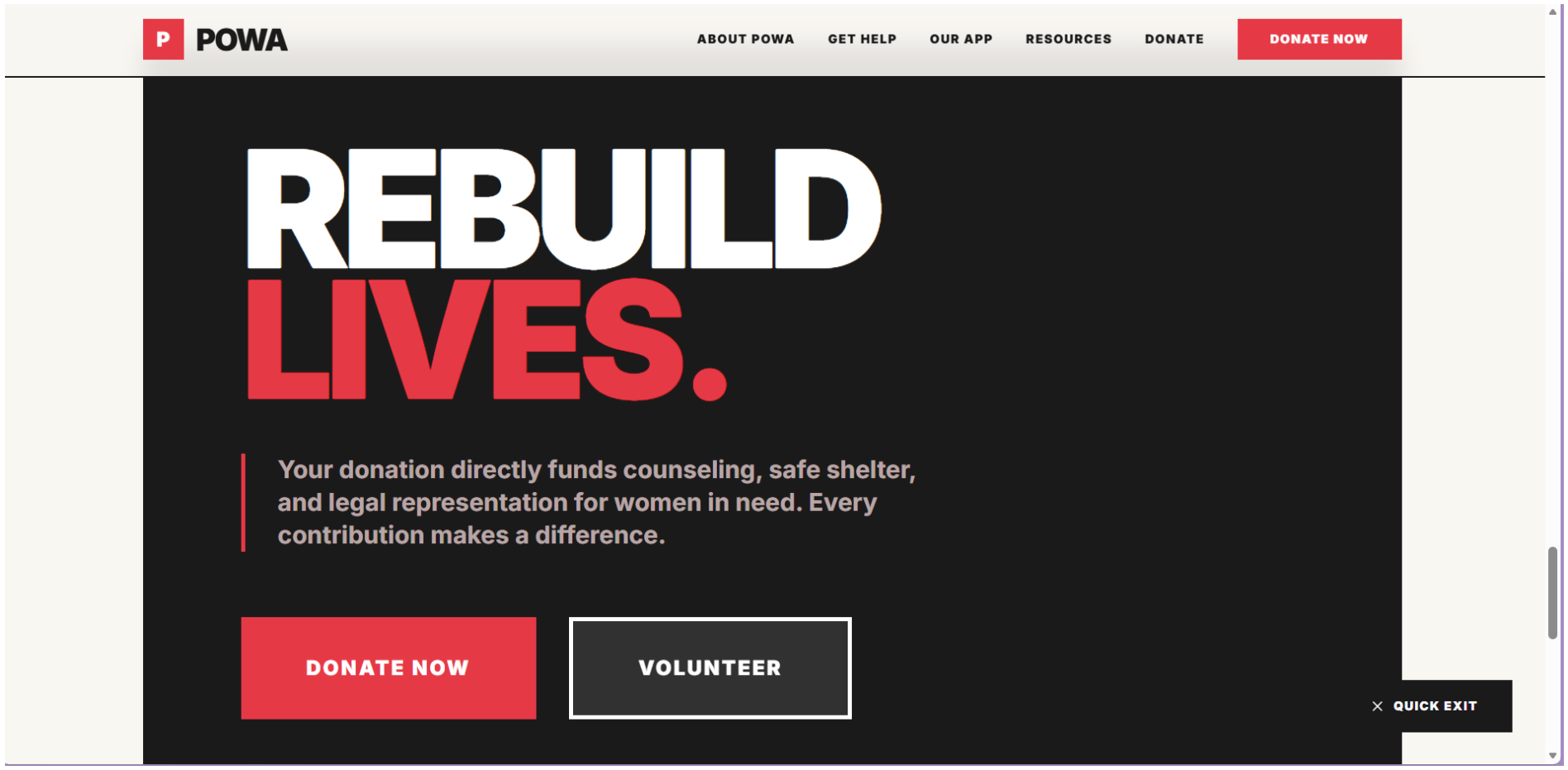
RESOURCES:

A call to action with a gloomy theme called "REBUILD LIVES." It offers two main buttons: "Donate Now" and "Volunteer." It also describes how donations support vital programs.



DONATE:

A call to action with a gloomy theme called "REBUILD LIVES." It offers two main buttons: "Donate Now" and "Volunteer." It also describes how donations support vital programs.



9.3. Accessibility Design Standards

The application interface must comply with WCAG 2.1 Level AA accessibility standards to ensure inclusivity for users with disabilities:

- All interactive elements must have a minimum touch target of 44x44 pixels.
- Colour contrast ratios must meet a minimum of 4.5:1 for normal text and 3:1 for large text.
- All images and icons must include descriptive alt text for screen reader compatibility.
- The SOS button must be accessible without scrolling and must be operable with one hand.
- Font size must be scalable, with a minimum of 16sp for body text.
- The application must support Android Talkback and iOS Voiceover screen reader technologies.

10. Database Design

10.1. Database Design Principles

The E.SOS App database is designed with the following principles to ensure security, performance, and POPIA compliance:

- Data minimization: Only information strictly necessary for system functionality is collected and stored.
- Encryption at rest: All sensitive fields are encrypted using AES-256.

- Access control: Database access is restricted to the application backend using the principle of least privilege.
- Data retention: Records are retained according to POPIA requirements and POWA's data governance policy.
- Referential integrity: Foreign key relationships ensure data consistency across related tables.

10.2. Database Tables

Table: users

Column	Data Type	Constraints	Description
user_id	VARCHAR(36)	PK, NOT NULL	UUID primary key
full_name	VARCHAR(100)	NOT NULL	Encrypted full name
phone_number	VARCHAR(20)	UNIQUE, NOT NULL	Encrypted phone number
email	VARCHAR(150)	UNIQUE, NOT NULL	Encrypted email address
password_hash	VARCHAR(255)	NOT NULL	bcrypt-hashed password
medical_info	TEXT	NULLABLE	Optional encrypted medical notes
is_active	BOOLEAN	DEFAULT TRUE	Account status flag
preferred_language	VARCHAR(10)	DEFAULT 'en'	Language preference (en, zu)
created_at	TIMESTAMP	NOT NULL	Account creation timestamp
updated_at	TIMESTAMP	NOT NULL	Last update timestamp

Table: sos_alerts

Column	Data Type	Constraints	Description
alert_id	VARCHAR(36)	PK, NOT NULL	UUID primary key
user_id	VARCHAR(36)	FK → users, NOT NULL	Owning user reference
alert_type	ENUM	NOT NULL	'SOS' or 'SILENT'
latitude	DECIMAL(10,8)	NOT NULL	GPS latitude at time of alert
longitude	DECIMAL(11,8)	NOT NULL	GPS longitude at time of alert
location_approximate	BOOLEAN	DEFAULT FALSE	True if GPS was unavailable
status	ENUM	NOT NULL	'ACTIVE', 'RESOLVED', 'CANCELLED'
notifications_sent	BOOLEAN	DEFAULT FALSE	Notification dispatch flag
triggered_at	TIMESTAMP	NOT NULL	UTC timestamp of activation
resolved_at	TIMESTAMP	NULLABLE	UTC timestamp of resolution
assigned_operator_id	VARCHAR(36)	FK → operators	Operator managing the alert

Table: incident_reports

Column	Data Type	Constraints	Description
report_id	VARCHAR(36)	PK, NOT NULL	UUID primary key
user_id	VARCHAR(36)	FK → users, NOT NULL	Reporting user
incident_type	VARCHAR(100)	NOT NULL	Category of abuse or incident
description	TEXT	NOT NULL	Encrypted incident description
incident_location	VARCHAR(255)	NULLABLE	Location string or address
date_occurred	TIMESTAMP	NOT NULL	When the incident occurred
media_file_urls	JSONB	NULLABLE	S3 URLs of attached media files

status	VARCHAR(50)	DEFAULT 'SUBMITTED'	Workflow status of the report
submitted_at	TIMESTAMP	NOT NULL	Report submission timestamp

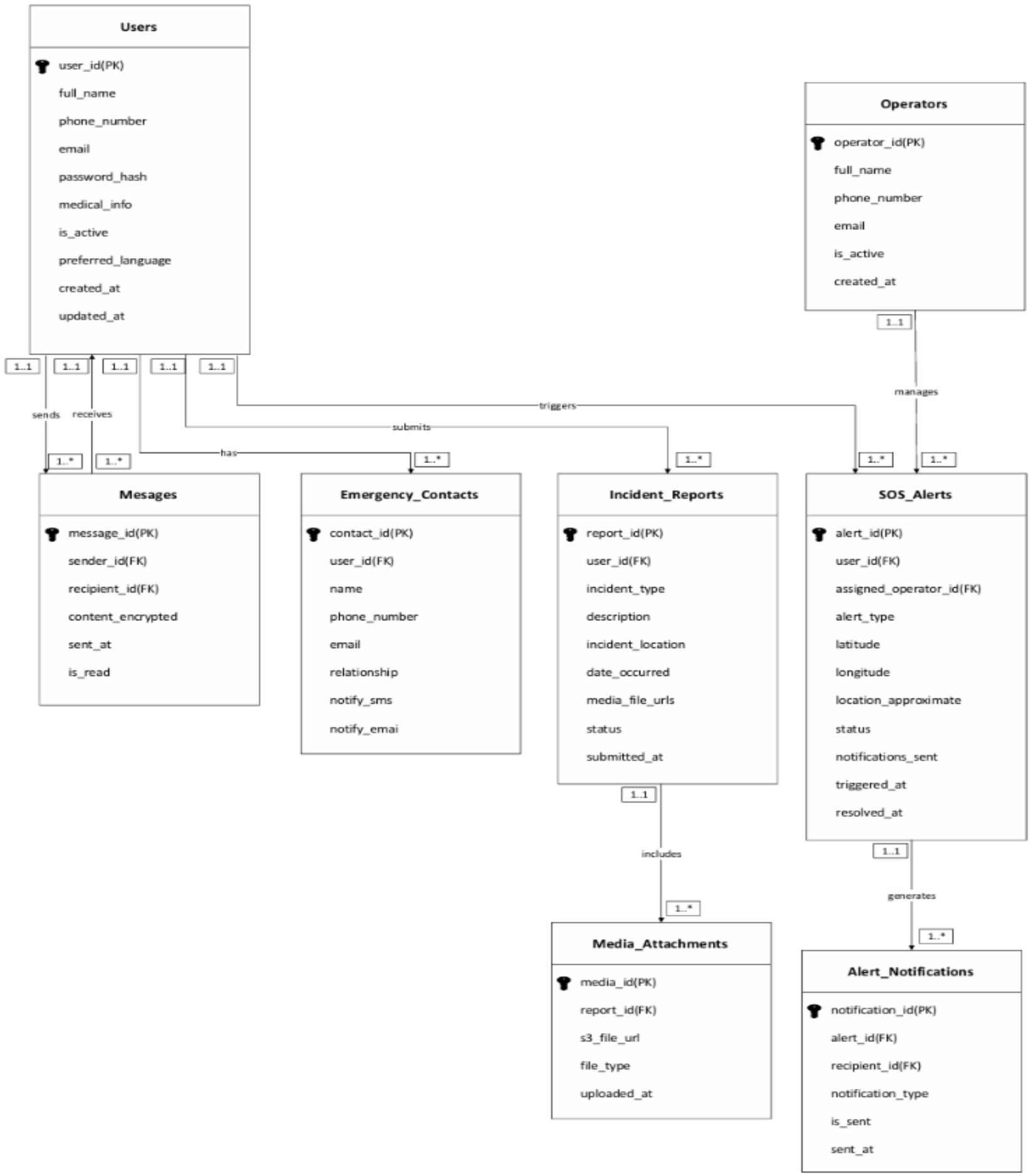
Table: emergency_contacts

Column	Data Type	Constraints	Description
contact_id	VARCHAR(36)	PK, NOT NULL	UUID primary key
user_id	VARCHAR(36)	FK → users, NOT NULL	Owning user
name	VARCHAR(100)	NOT NULL	Contact's full name
phone_number	VARCHAR(20)	NOT NULL	Encrypted mobile number
email	VARCHAR(150)	NULLABLE	Optional encrypted email
relationship	VARCHAR(50)	NOT NULL	Relationship to user
notify_sms	BOOLEAN	DEFAULT TRUE	Receive SMS notifications
notify_email	BOOLEAN	DEFAULT FALSE	Receive email notifications

Table: messages

Column	Data Type	Constraints	Description
message_id	VARCHAR(36)	PK, NOT NULL	UUID primary key
sender_id	VARCHAR(36)	NOT NULL	Sender user_id or operator_id
recipient_id	VARCHAR(36)	NOT NULL	Recipient user_id or operator_id
content_encrypted	TEXT	NOT NULL	End-to-end encrypted message body
sent_at	TIMESTAMP	NOT NULL	Message send timestamp
is_read	BOOLEAN	DEFAULT FALSE	Read receipt flag

11. Entity Relationship Database Diagram



12. System Report Design

The E.SOS App provides automated and on-demand reporting features to support operational management, incident monitoring, performance evaluation, and strategic decision-making within POWA. The reports assist management, operators, administrators, and compliance personnel in monitoring system activities, analysing emergency response trends, and improving service delivery.

Reports are generated through the web dashboard and can be exported in formats such as PDF and CSV for record-keeping, analysis, and reporting purposes (Eckerson, 2010).

System Reports

#	Report Name	Audience	Data Fields	Frequency
R-01	**Alert Activity Report**	POWA Management	Total number of alerts, alert categories, response times, resolution status, and operator activities	Monthly / On-demand
R-02	**Incident Report Summary**	POWA Operators / Legal Team	Incident types, report dates, locations, case status, assigned operators, and incident outcomes	Weekly / On-demand
R-03	**Geographic Hotspot Analysis**	POWA Management / Advocacy	GPS-based alert locations, regional alert distribution, and hotspot frequency analysis	Monthly
R-04	**Response Time Performance Report**	POWA Management	Average response time, operator acknowledgement time, and peak emergency activity periods	Monthly
R-05	**User Registration Report**	System Administrator	New user registrations, active accounts, deactivated users, and device platform statistics	Monthly
R-06	**Audit and Security Log Report**	System Administrator / Compliance	Login history, failed authentication attempts, user activity logs, and administrative actions	Monthly / On-demand
R-07	**POPIA Compliance Report**	POWA Data Officer / Legal	Consent records, data access requests, retention compliance, and data management activities	Quarterly

Mobile App and Website Development (Task 2)

1. Introduction

The E.SOS project is an emergency support system designed for People Opposing Women Abuse (POWA). The system aims to assist individuals experiencing gender-based violence, domestic abuse, harassment, or other emergency situations by providing accessible ways to request assistance and access support services.

The project involves developing a mobile application and a website to support users in emergency situations. The system is intended to improve access to assistance while considering user safety, privacy, and security.

This report documents the development of the E.SOS mobile application and website. It covers the implemented features, database design and implementation, authentication, testing, and DevOps processes. It will also include screenshots and other evidence to demonstrate the progress and functionality of the system.

2. Project Overview

2.1 Background of the Project

E.SOS builds on the requirements analysis and system design completed during the previous stage of the WIL project. The system is being developed to provide users with digital access to emergency support services associated with POWA.

The planned features include user registration and login, emergency SOS alerts, location sharing, incident reporting, messaging, and access to support information. The mobile application and website are intended to work together to provide access to the system's services.

2.2 Project Objectives

The objectives of the E.SOS project are to:

1. Develop an Android mobile application using Kotlin.
2. Develop a website that supports the E.SOS system.
3. Implement the required authentication features, including Single Sign-On (SSO).
4. Create and populate a relational database to store system information.
5. Implement appropriate input validation and error handling.
6. Configure an automated build pipeline and automated unit tests.
7. Provide documentation to guide users.
8. Test the implemented features and document the results using screenshots and supporting evidence.

3. Project Repositories and Media Links

3.1 GitHub Repository Link

GitHub Link: https://github.com/Sheketli/XISD6329_POE_TaskTwo-E.SOS-App.git

Azura DevOps Git Repo for Mobile App link: https://dev.azure.com/ST10452756-XISD6321POE/E.SOS%20Project%20Software%20Application/_git/E.SOS%20Mobile%20App%20Project%20Software%20Application%20Repo

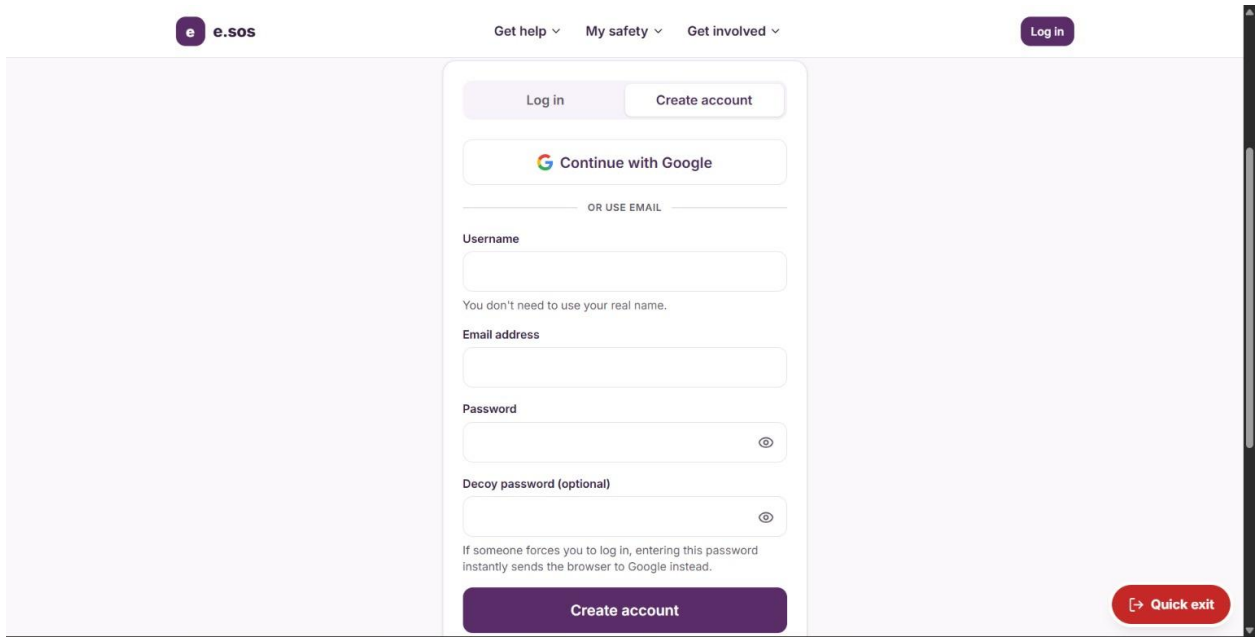
Azura DevOps Git Repo for Website App link: https://dev.azure.com/ST10452756-XISD6321POE/E.SOS%20Project%20Software%20Application/_git/E.SOS%20Website%20App%20Project%20Software%20Application%20Repo

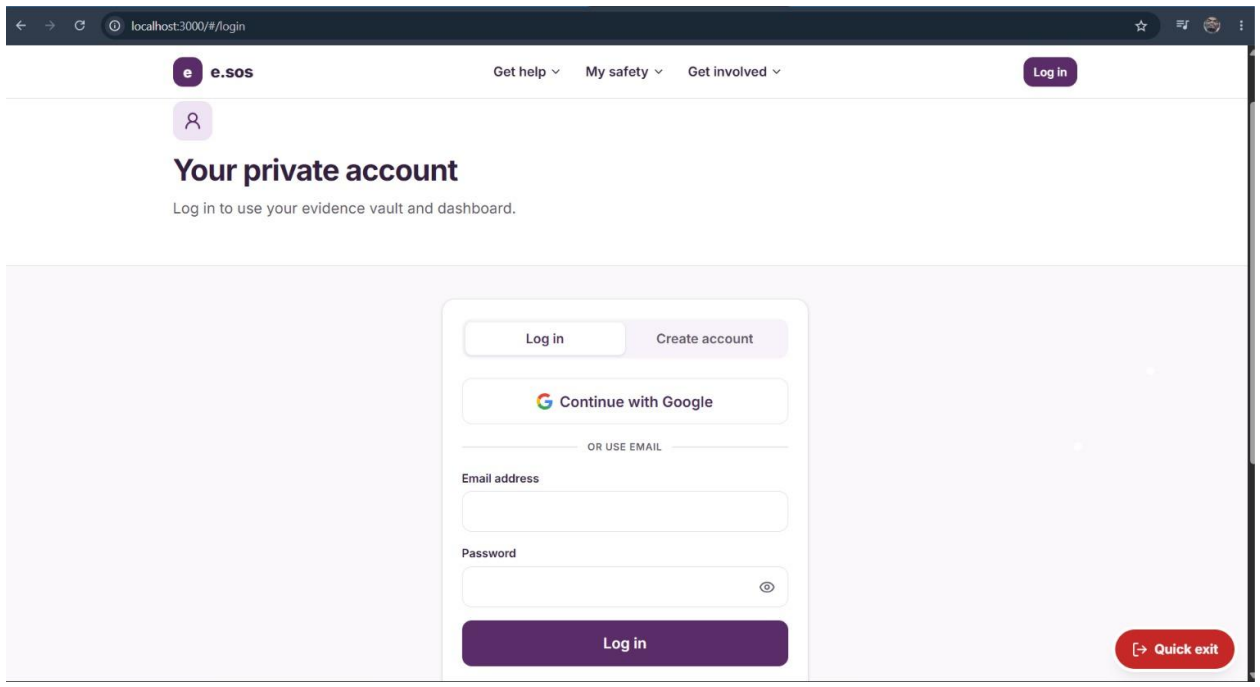
3.2 Demonstration Video Link

E.SOS Video Presentation link: https://youtu.be/uFZPVgoJxNo?si=g-nlOk9T8iaSSQNn

4. Mobile Application Interface and Features

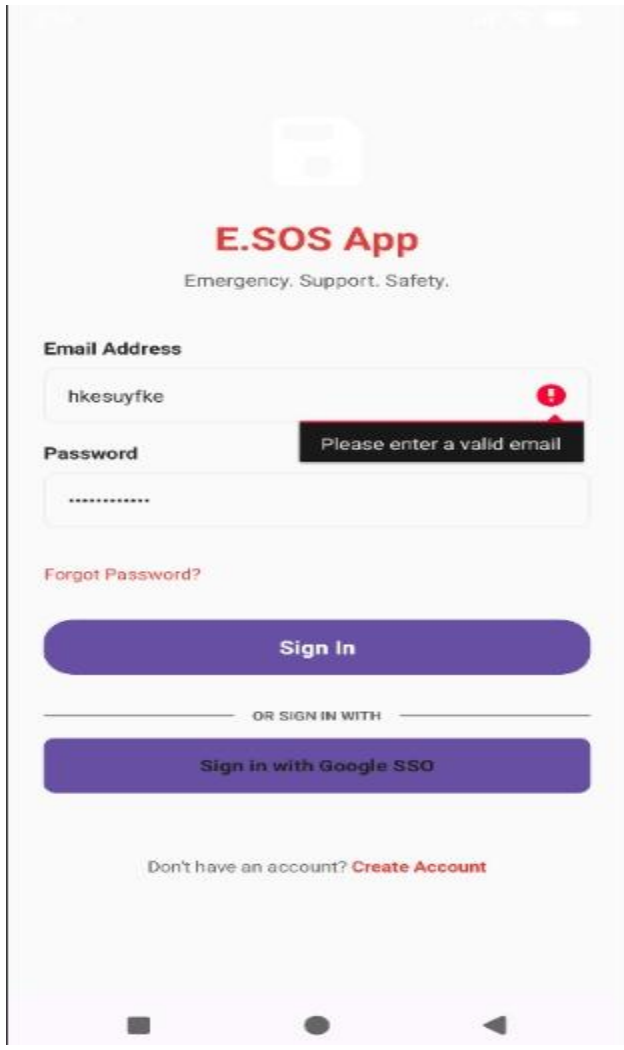
4.1 User Registration and Login





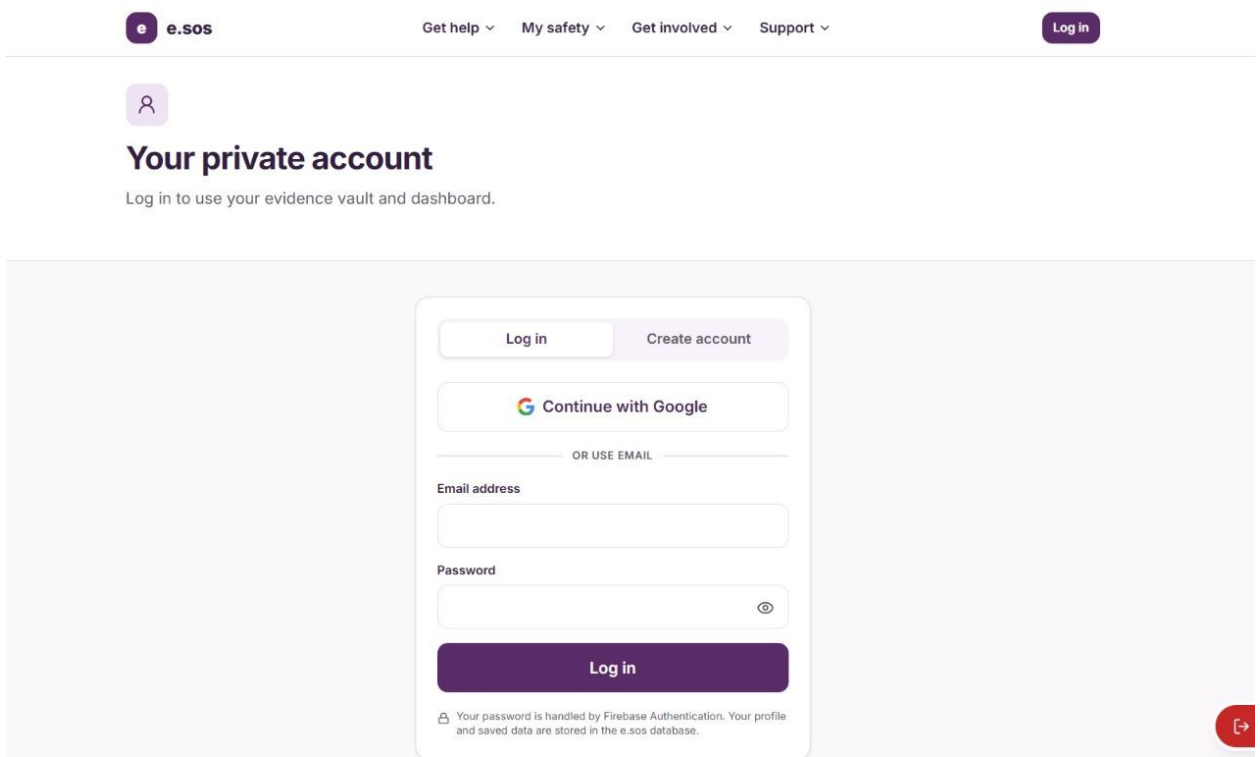
A new user opens the E.SOS website and selects the Create account tab, where they can sign up instantly with Continue with Google or register with email by entering a username, an email address and a password. They may also set an optional decoy password, which redirects the browser to Google if someone forces them to log in. After registering, the user returns to the Log in tab and signs in with their email and password, or with Google, and is taken to their private dashboard and evidence vault. To protect user privacy, passwords are encrypted securely before being saved. Each password is processed with a salted PBKDF2 algorithm (210,000 iterations), so the database stores only unreadable value and never the original password. Input validation and clear error messages guide the user if any detail is missing or incorrect.

Input Validation and Error Handling

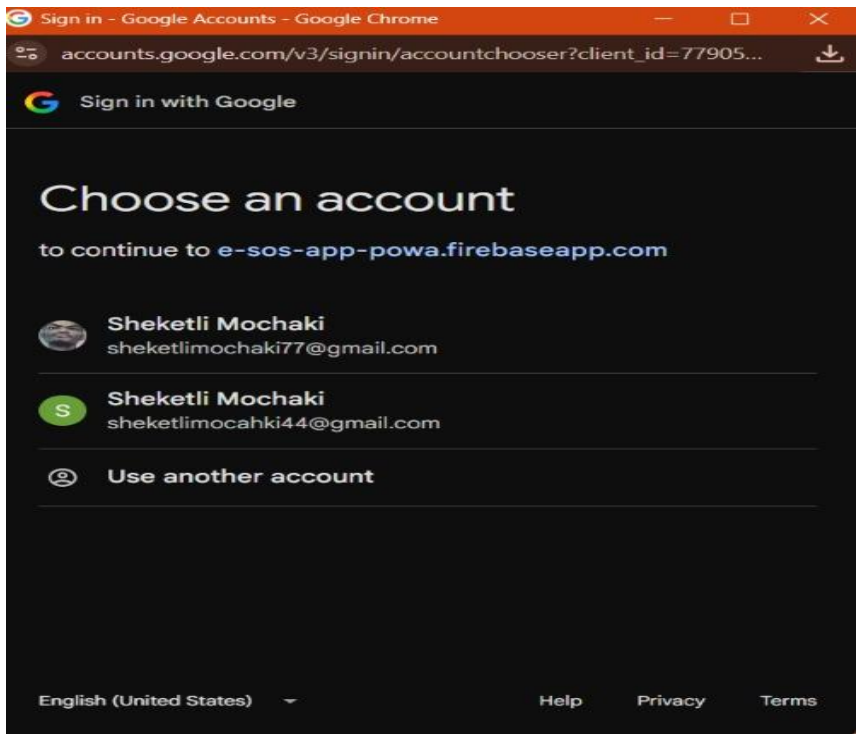


To prevent invalid data from causing errors or crashes, the E.SOS mobile app validates user input before it is processed. Figure X shows the login screen after a user typed text that is not an email address (“hkesuyfke”) into the Email Address field. Instead of crashing or failing silently, the app displays a red warning icon and a clear message, “Please enter a valid email”, telling the user exactly what to correct. The user can fix the entry and try again without losing their place in the app. This keeps the app stable and user-friendly, and it also stops invalid data from being sent to the authentication system.

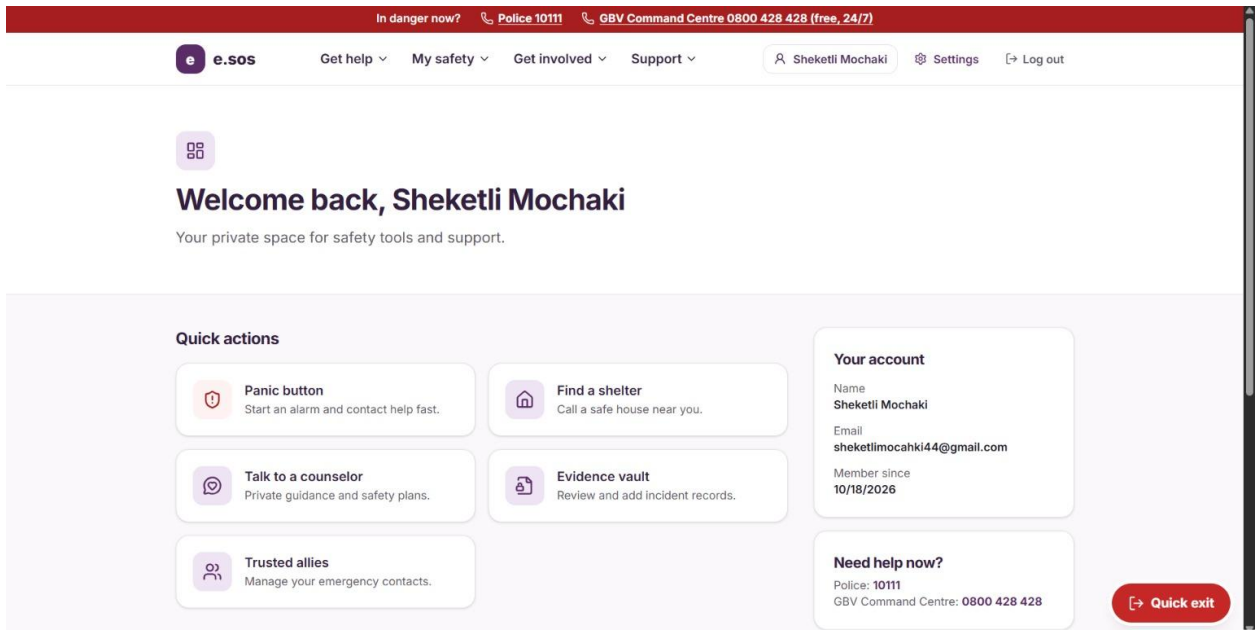
4.2 Single Sign-On (SSO) Implementation



E.SOS integrates **Google Sign-In** as its third-party identity provider, implemented through **Firebase Authentication**. On both the website and the mobile app, a user can sign in with a single click by selecting **Continue with Google** (website) or **Sign in with Google SSO** (mobile app). The user is redirected to Google, chooses their account, and is returned to E.SOS already signed in, with no separate password to create or remember. Once authenticated, Firebase issues a secure session token that identifies the user. The website and the mobile app both use the same Firebase project, so a Google account that signs in on one platform is recognised on the other as the same user. This token is checked each time the user opens a protected area such as their dashboard and evidence vault, and when the user logs out the session ends and access is removed. Profile data linked to the account is stored in the e.sos database, where the auth_provider column records whether the user registered with Google or with email and password.

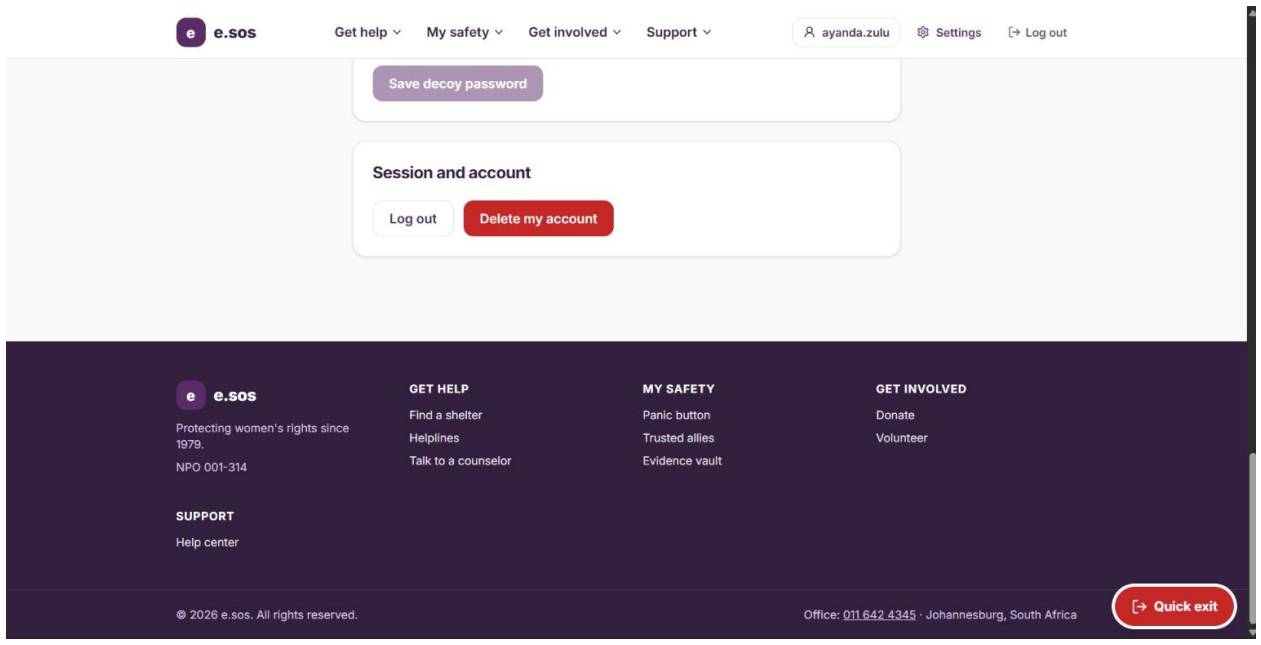


After the user selects **Continue with Google**, a Google sign-in window opens asking them to choose which account to use for E.SOS The window is hosted by Google at accounts.google.com, so the user's Google credentials are entered on Google's own page and are never seen or stored by E.SOS. Once the user selects an account, Google confirms their identity to Firebase Authentication, and the user is returned to E.SOS signed in.

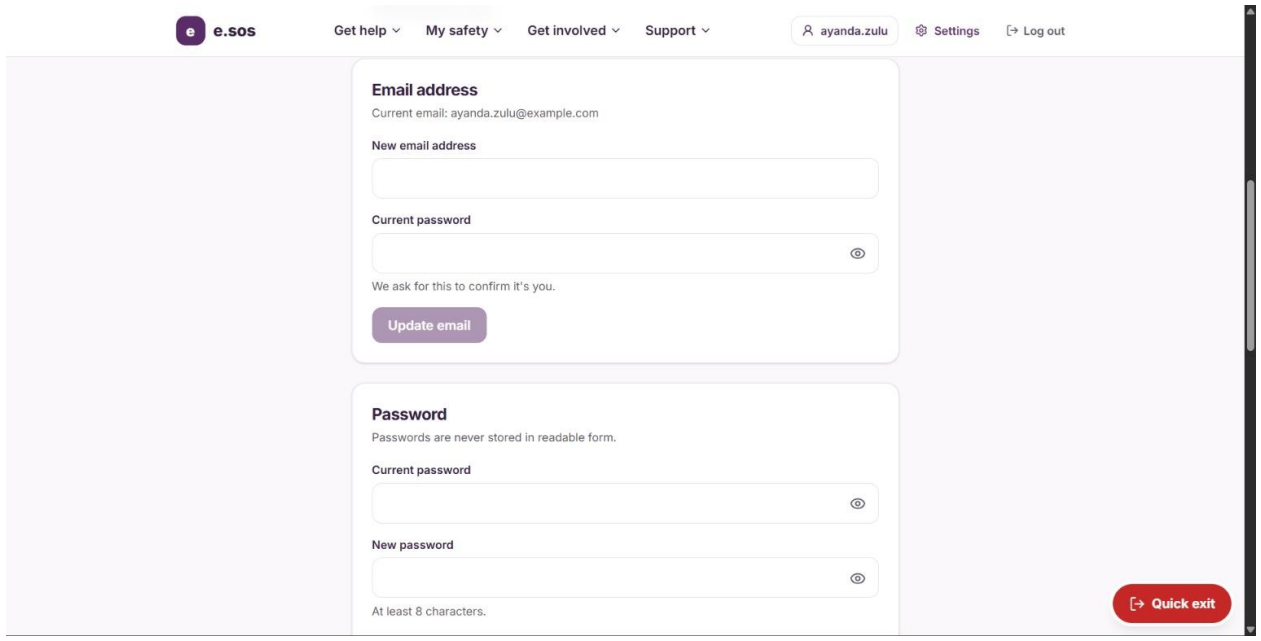


After selecting an account in the Google pop-up, the user is redirected into the E.SOS dashboard. The page greets the user by name, “Welcome back, Sheketli Mochaki”, and the Your account card shows the name and email taken from the Google account, along with the date the member joined. The user is now signed in, as shown by their name, Settings and Log out in the top navigation bar, and can reach the safety tools under Quick actions: Panic button, Find a shelter, Talk to a counselor, Evidence vault and Trusted allies. Emergency numbers (Police 10111 and the GBV Command Centre 0800 428 428) are shown in the red banner at the top and in the Need help now? card.

4.3 User Settings and Password Management

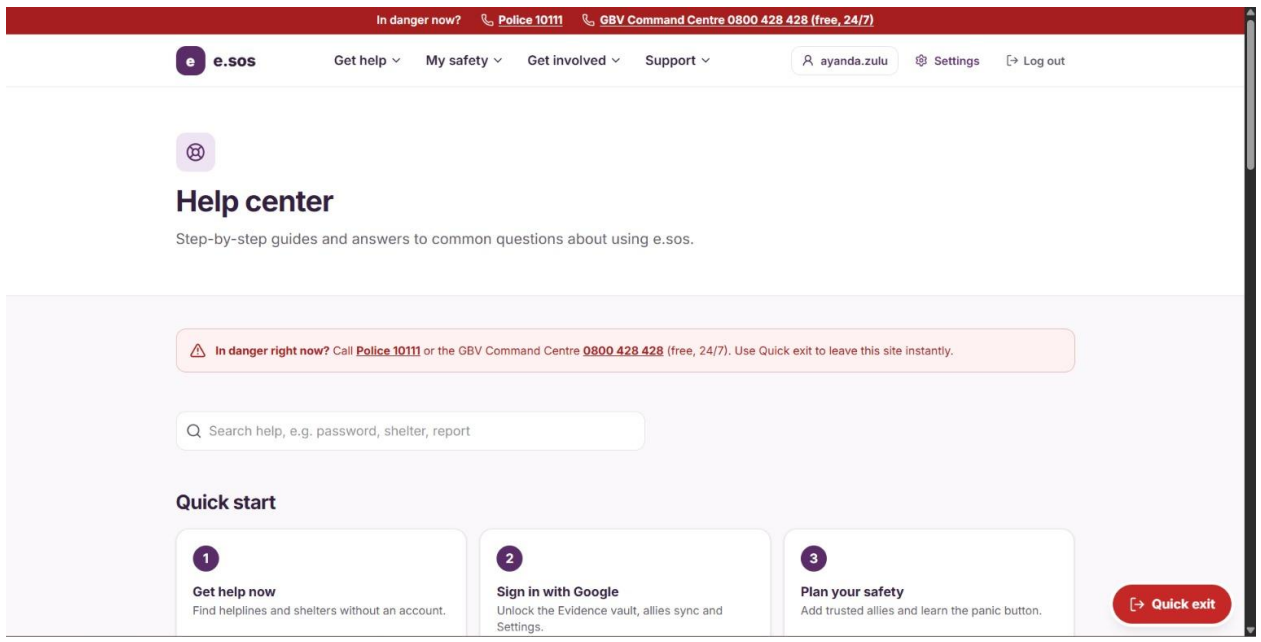


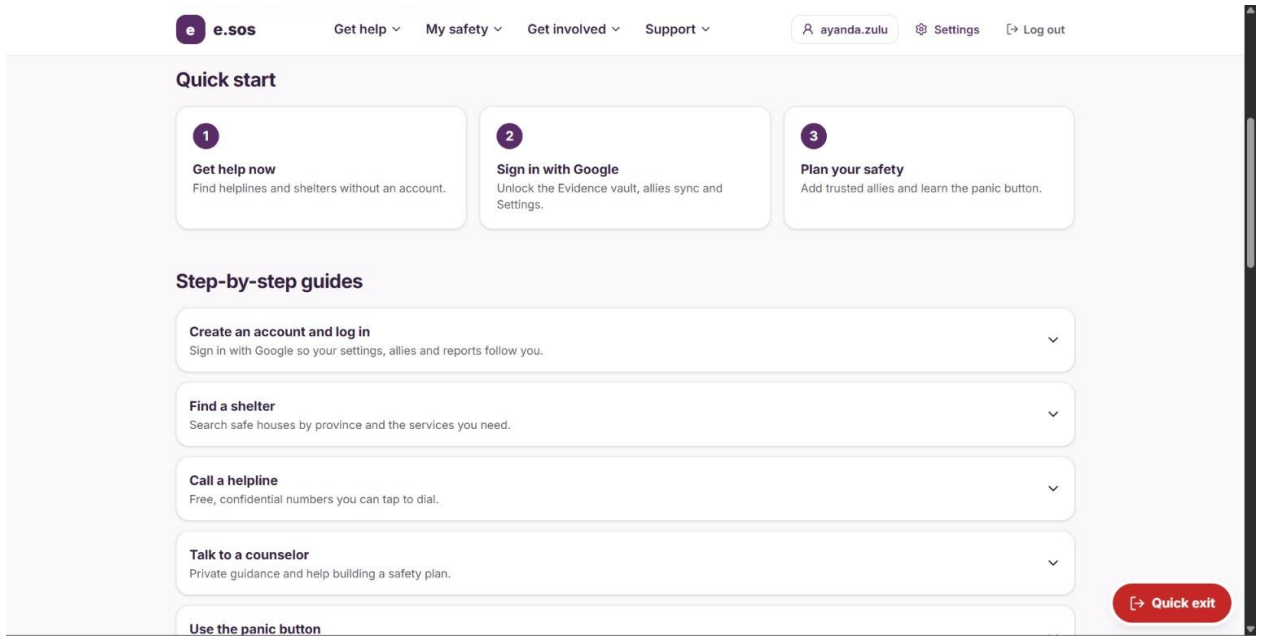
E.SOS gives users control over their own accounts through a Settings page, which a logged-in user opens by selecting Settings in the top navigation bar next to their username. From here the user can manage their security and account options in one place. To change their security details, the user enters the new value in the relevant field and selects the save button, for example Save decoy password to update the optional decoy password used for emergencies. The new password is hashed securely before it is stored, so the original is never saved in plain text. A Session and account section lets the user end their session by selecting Log out, or permanently remove their data by selecting Delete my account, which is shown in red to warn the user that the action is serious. Users who signed in with Google manage their main password through their Google account, since E.SOS never handles those credentials. A Quick exit button remains visible on every page so a user in danger can leave the site immediately.



This page shows the **Settings** page of the E.SOS website for the logged-in user ayanda.zulu. The page is split into separate cards. The **Email address** card displays the user's current email and lets them enter a **New email address**, with their **Current password** required to confirm it's really them, before selecting **Update email**. Below it, the **Password** card states that passwords are never stored in readable form and provides **Current password** and **New password** fields, each with an eye icon to show or hide the text, and a note that the new password must be at least 8 characters. The buttons appear faded because the fields are still empty. The **Quick exit** button stays visible in the bottom-right corner.

4.4 Help Documentation and User Support





E.SOS includes a built-in Help center to support users who get stuck or confused, reached from the Support menu in the navigation bar . The page opens with a short description, “Step-by-step guides and answers to common questions about using e.sos”, and a prominent emergency notice that gives the Police (10111) and GBV Command Centre (0800 428 428) numbers and reminds users they can leave instantly with Quick exit. Users can type a keyword such as “password”, “shelter” or “report” into the search bar to find an answer quickly. A Quick start section offers three numbered steps: *Get help now* (find helplines and shelters without an account), *Sign in with Google* (unlock the Evidence vault, allies sync and Settings), and *Plan your safety* (add trusted allies and learn the panic button). Below this, Step-by-step guides cover the main tasks, including creating an account and logging in, finding a shelter, calling a helpline, talking to a counselor and using the panic button. Each guide expands when selected, so users see only the instructions they need. The language is simple and the layout uncluttered, which suits users who may be stressed or in a hurry.

5. Database Design and Verification

5.1 Relational Database Structure and Tables

Database Management System

The e.sos platform uses **SQLite**, a lightweight, file-based relational DBMS. It needs no separate database server, which suits a project that must run on both web and mobile clients. The schema is defined in `database/schema.sql` and created with the `npm run db:init` script. The script enables `PRAGMA foreign_keys = ON`, so SQLite enforces referential integrity. Every table uses `CREATE TABLE IF NOT EXISTS`, so the script can be re-run safely. Data integrity is further protected with `NOT NULL`, `UNIQUE`, `DEFAULT` and `CHECK` constraints. For example, `role` is limited to `member`, `volunteer` or `admin`, shelter `capacity` to `Available`, `Limited` or `Full`, and `amount_zar` must be greater than 0.

Table Structure, Primary Keys and Foreign Keys

The database has nine tables and one view, in three groups.

1. User-centred tables (linked by foreign keys)

Table	Primary Key	Foreign Key	Relationship
users	Id (TEXT,Firebase UID)	**none**	Parent table for user-owned data
evidence_reports	Id (TEXT)	user_id > user (id)	One user has many reports
Report_aatachments	Id (INTEGER, AUTONCREMENT)	report_*id* > *evidence*_report(id)	One report has many attachments
Trusted_allies	Id (TEXT)	User_id >user(id)	One user has many trusted contacts

All three foreign keys use `ON DELETE CASCADE`. If a user account is deleted, their reports, the attachments of those reports, and their trusted allies are deleted automatically, leaving no orphaned records. This matters for a safety platform that stores sensitive evidence. The `users` table also has a unique, case-insensitive `email` and stores only PBKDF2-SHA256 password hashes (`password_hash`, plus `decoy_hash` for the optional decoy password), never plaintext passwords.

2. Shelter directory (one-to-many)

- `shelters` (PK `id`) holds each shelter's name, location, province, phone and capacity status.
- `shelter_services` (PK `id`, FK `shelter_id` → `shelters(id)`, `ON DELETE CASCADE`) lists the services each shelter offers. A `UNIQUE (shelter_id, service)` constraint prevents duplicate services for one shelter.

3. Standalone tables (no foreign keys)

- `helplines` (PK `id`, unique `name`) stores emergency and support numbers, with an `is_top` flag for featured helplines.
- `donations` (PK `id`, unique `reference`) records donor details, the amount in ZAR and a status of `pledged`, `received` or `cancelled`.
- `volunteer_applications` (PK `id`) tracks applicants through the statuses `new`, `screening`, `accepted` and `declined`.

These tables do not depend on a logged-in user, so they can be used by the public.

Indexes and view. Four indexes (`idx_reports_user`, `idx_allies_user`, `idx_services_shelter`, `idx_attachments_report`) sit on the foreign key columns to speed up joins and lookups. The view `v_users` gives a readable way to browse user records in VS Code. It exposes only one-way hashes, never plaintext secrets.

Data Flow Between the Database and the User Interface

The backend API sits between the user interface (web and mobile) and the SQLite database. The client never queries the database directly. The user logs in and Firebase issues a UID, which is stored as `users.id`. The UI then sends requests to the API (for example, submitting an incident report or adding a trusted ally). The API validates the request and runs parameterised SQL against the matching tables, using the UID to scope data to the right user. For example, it inserts a row into `evidence_reports` and then rows into `report_attachments` for each uploaded file. Query results are returned to the UI as JSON, which the interface renders as reports, shelter listings or helpline lists. Public data such as shelters, helplines, donations and volunteer applications is served through the same API without requiring a user-owned record.

Note: I wrote the API paragraph from the schema alone. Please add your actual backend framework (e.g. Express/Node.js) and library (e.g. better-sqlite3) so it matches your code.

Screenshots: SQL Table Definitions

Figure 3.1a. `schema.sql` lines 1–37: the `users`, `shelters` and `shelter_services` tables.

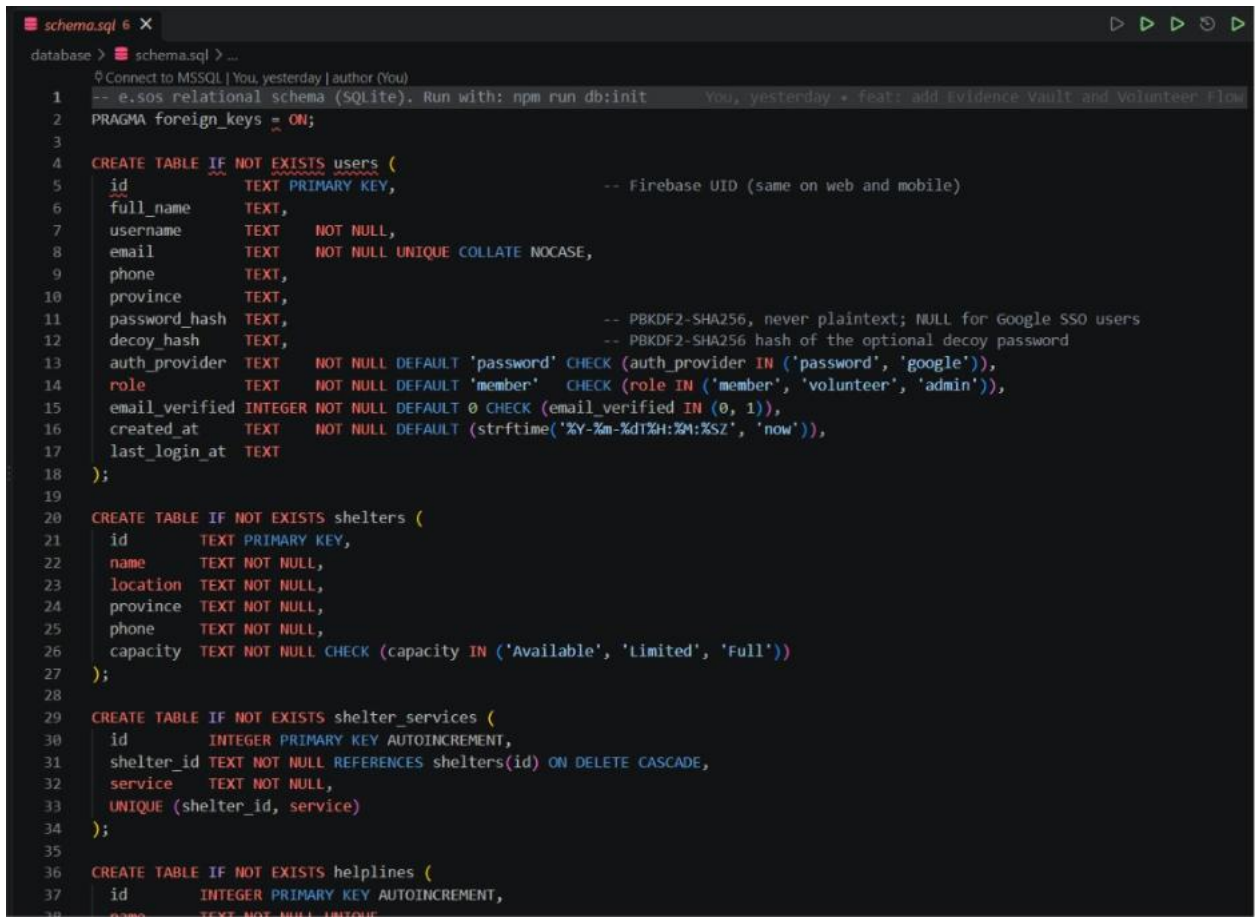


Figure 3.1b. `schema.sql` lines 36–71: the `helplines`, `evidence_reports`, `report_attachments` and `trusted_allies` tables.

```sql
CREATE TABLE IF NOT EXISTS helplines (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  name      TEXT NOT NULL UNIQUE,
  phone     TEXT NOT NULL,
  dial      TEXT NOT NULL,
  category  TEXT NOT NULL,
  details   TEXT NOT NULL,
  is_top    INTEGER NOT NULL DEFAULT 0 CHECK (is_top IN (0, 1))
);

CREATE TABLE IF NOT EXISTS evidence_reports (
  id              TEXT PRIMARY KEY,
  user_id         TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  title           TEXT NOT NULL,
  incident_type   TEXT NOT NULL,
  incident_date   TEXT NOT NULL,                          -- YYYY-MM-DD
  incident_time   TEXT NOT NULL,                          -- HH:MM
  description     TEXT NOT NULL,
  confidentiality TEXT NOT NULL DEFAULT 'private',
  created_at      TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ', 'now'))
);

CREATE TABLE IF NOT EXISTS report_attachments (
  id        INTEGER PRIMARY KEY AUTOINCREMENT,
  report_id TEXT NOT NULL REFERENCES evidence_reports(id) ON DELETE CASCADE,
  file_name TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS trusted_allies (
  id           TEXT PRIMARY KEY,
  user_id      TEXT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  name         TEXT NOT NULL,
  relationship TEXT NOT NULL,
  phone        TEXT NOT NULL,
  created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ', 'now'))
);
```

Figure 3.1c. `schema.sql` lines 70–104: the `donations` and `volunteer_applications` tables, the four indexes and the `v_users` view.

```sql
  created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ', 'now'))
);

CREATE TABLE IF NOT EXISTS donations (
  id          INTEGER PRIMARY KEY AUTOINCREMENT,
  reference   TEXT NOT NULL UNIQUE,
  donor_name  TEXT,
  donor_email TEXT NOT NULL,
  amount_zar  REAL NOT NULL CHECK (amount_zar > 0),
  status      TEXT NOT NULL DEFAULT 'pledged' CHECK (status IN ('pledged', 'received', 'cancelled')),
  created_at  TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ', 'now'))
);

CREATE TABLE IF NOT EXISTS volunteer_applications (
  id           INTEGER PRIMARY KEY AUTOINCREMENT,
  full_name    TEXT NOT NULL,
  email        TEXT NOT NULL,
  phone        TEXT,
  role         TEXT NOT NULL,
  availability TEXT NOT NULL,                             -- comma-separated list
  motivation   TEXT NOT NULL,
  status       TEXT NOT NULL DEFAULT 'new' CHECK (status IN ('new', 'screening', 'accepted', 'declined')),
  created_at   TEXT NOT NULL DEFAULT (strftime('%Y-%m-%dT%H:%M:%SZ', 'now'))
);

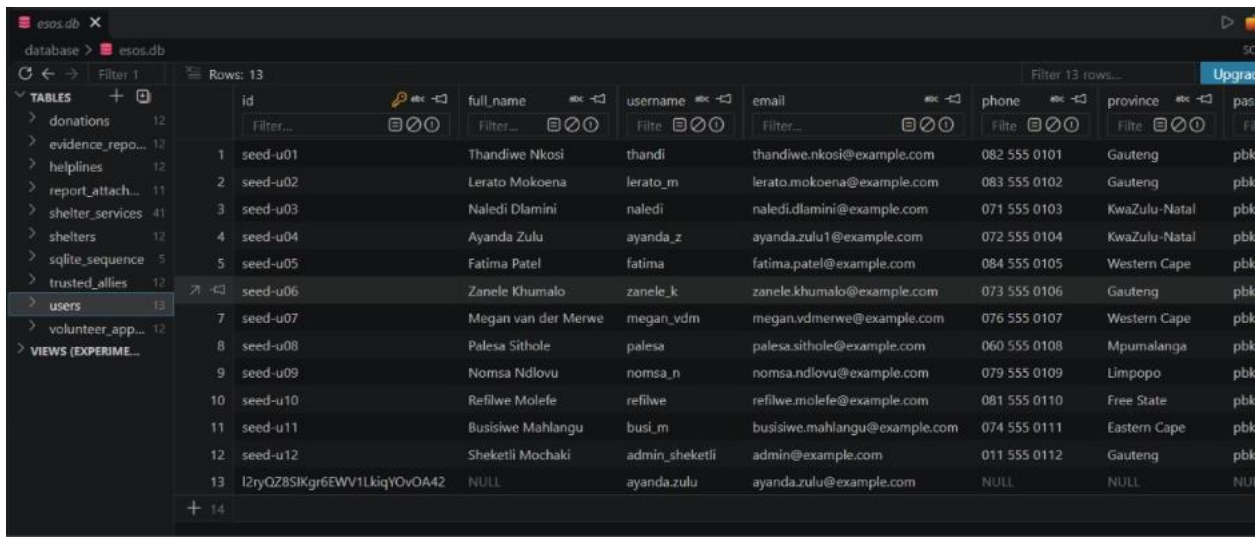
CREATE INDEX IF NOT EXISTS idx_reports_user ON evidence_reports(user_id);
CREATE INDEX IF NOT EXISTS idx_allies_user ON trusted_allies(user_id);
CREATE INDEX IF NOT EXISTS idx_services_shelter ON shelter_services(shelter_id);
CREATE INDEX IF NOT EXISTS idx_attachments_report ON report_attachments(report_id);

-- Readable view for browsing users in VS Code (the only secrets are one-way hashes)
CREATE VIEW IF NOT EXISTS v_users AS
SELECT id, full_name, username, email, phone, province, auth_provider, role, email_verified,
       password_hash, decoy_hash, created_at, last_login_at
FROM users;
```

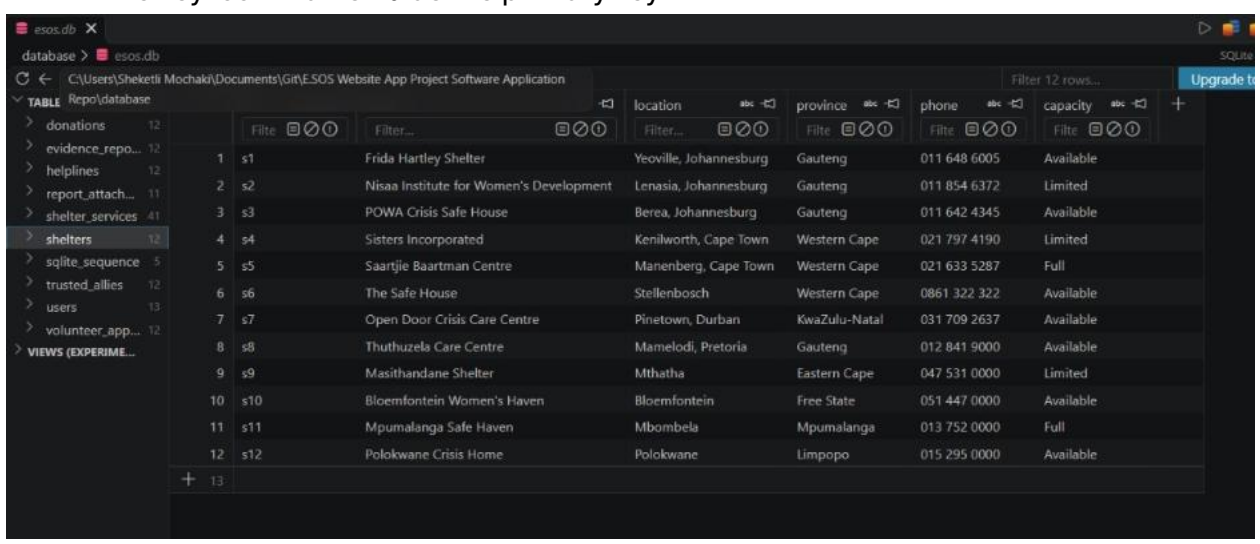
ERD: still to add. Draw it from the relationships above: `users` 1—M `evidence_reports` 1—M `report_attachments`, `users` 1—M `trusted_allies`, and `shelters` 1—M `shelter_services`, with `helplines`, `donations` and `volunteer_applications` shown standalone.

5.2 Database Population and Verification

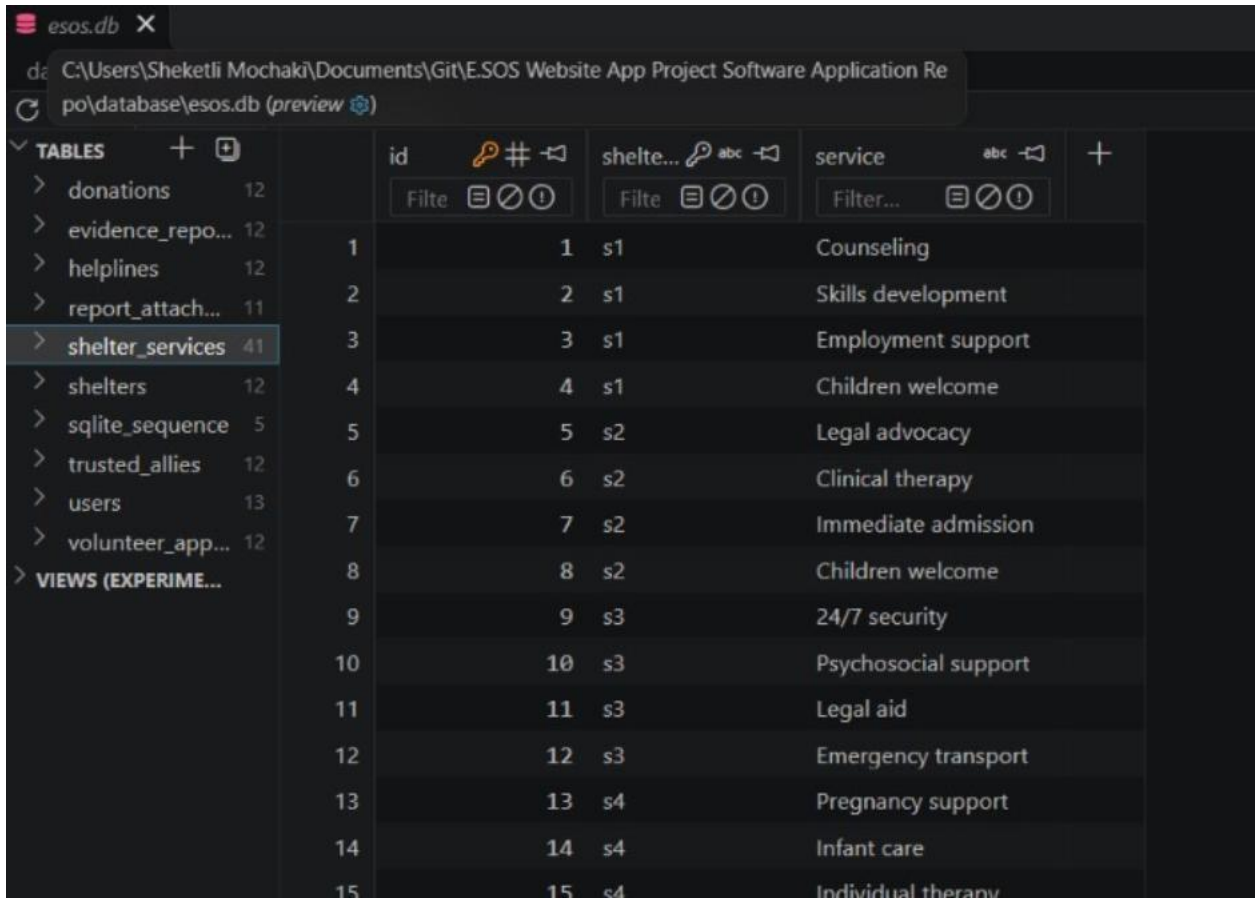
The system was populated with simulated records to test how the database handles real data. Every one of the nine tables holds at least 10 rows, which meets the rubric requirement. The records were viewed in the SQLite Viewer extension in VS Code, opened on `esos.db`.



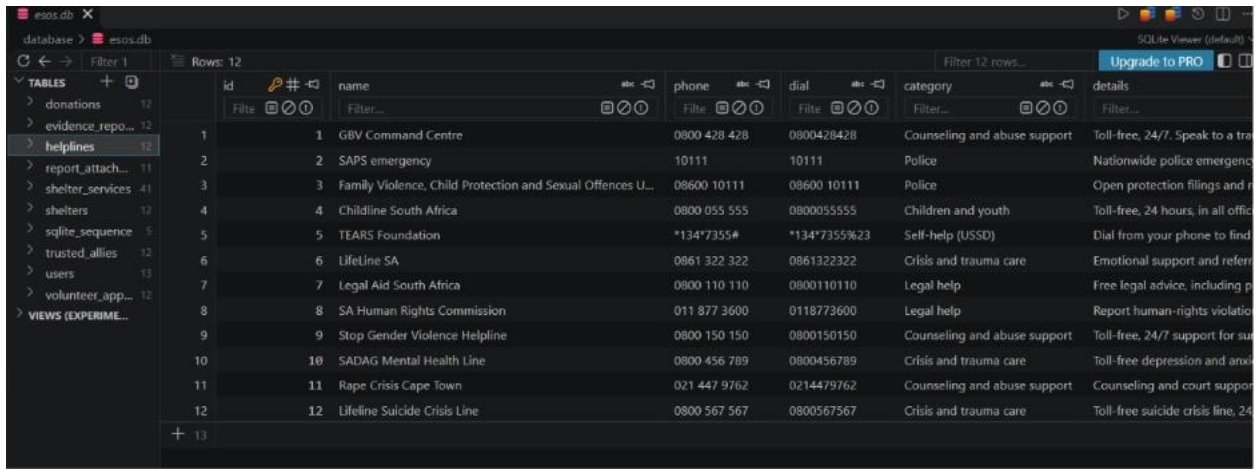
- Rows `seed-u01` to `seed-u12` are test accounts with names, usernames, emails and provinces.
- Row 13 has a Firebase-style ID and empty profile fields.
- The key icon marks `id` as the primary key.



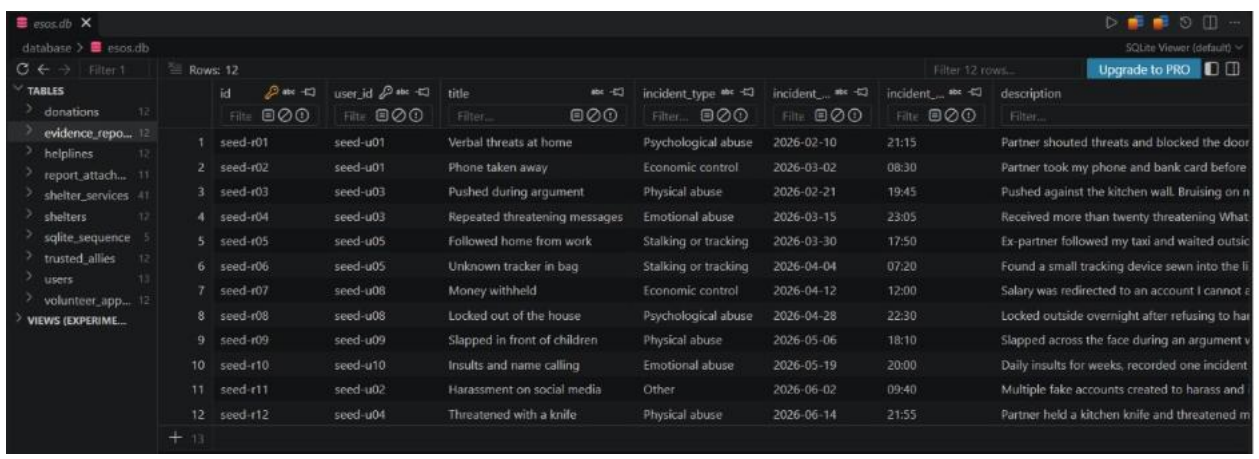
Each shelter has an ID (`s1` to `s12`), a name, a location, a province, a phone number and a capacity of Available, Limited or Full.



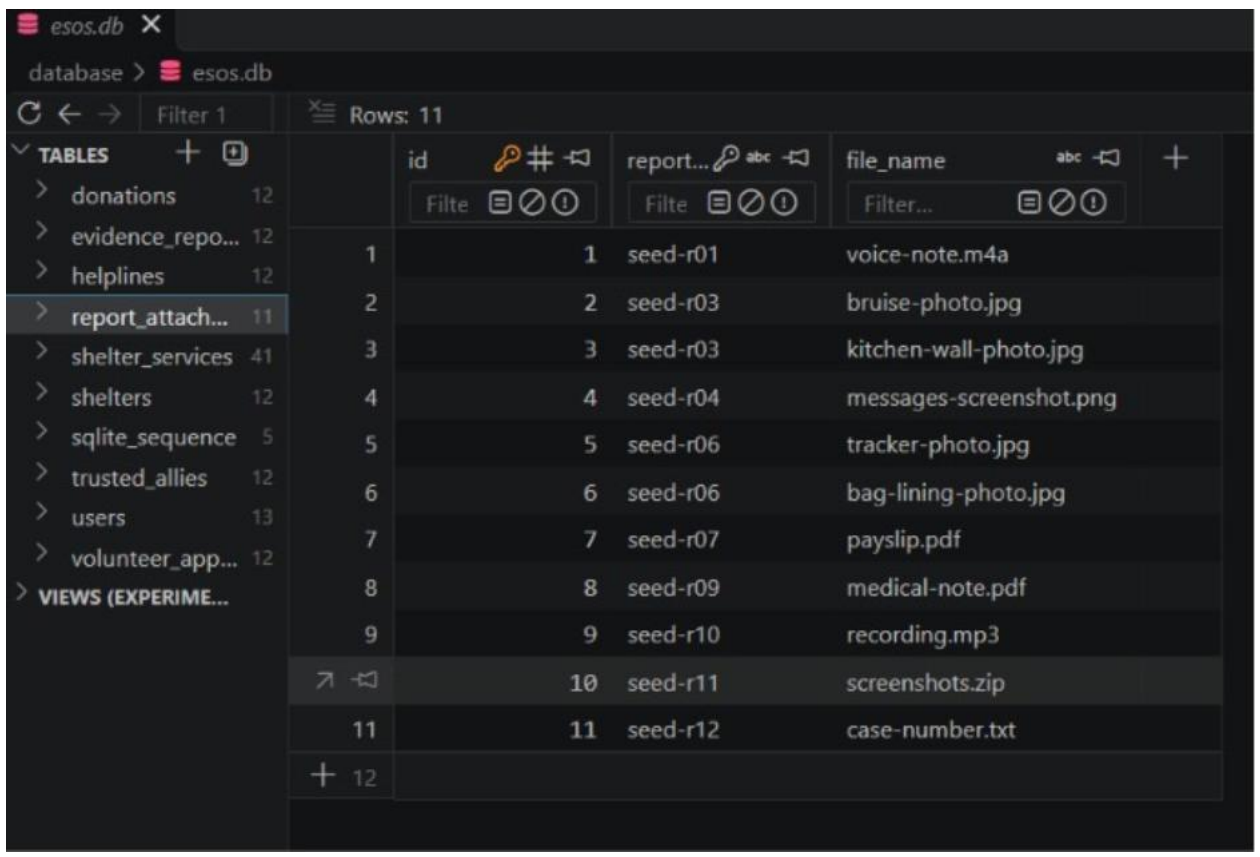
- Each row is one service offered by one shelter. `shelter_id` (`s1`, `s2` and so on) matches the `shelters` table.
- One shelter has several rows, for example `s1` offers Counseling, Skills development and more. This is the one-to-many link.



- These are real South African support lines, such as GBV Command Centre, SAPS and Childline.
- phone is the display format and `dial` is the digits-only version for tap-to-call. The `category` column groups them, for example Police or Legal help.

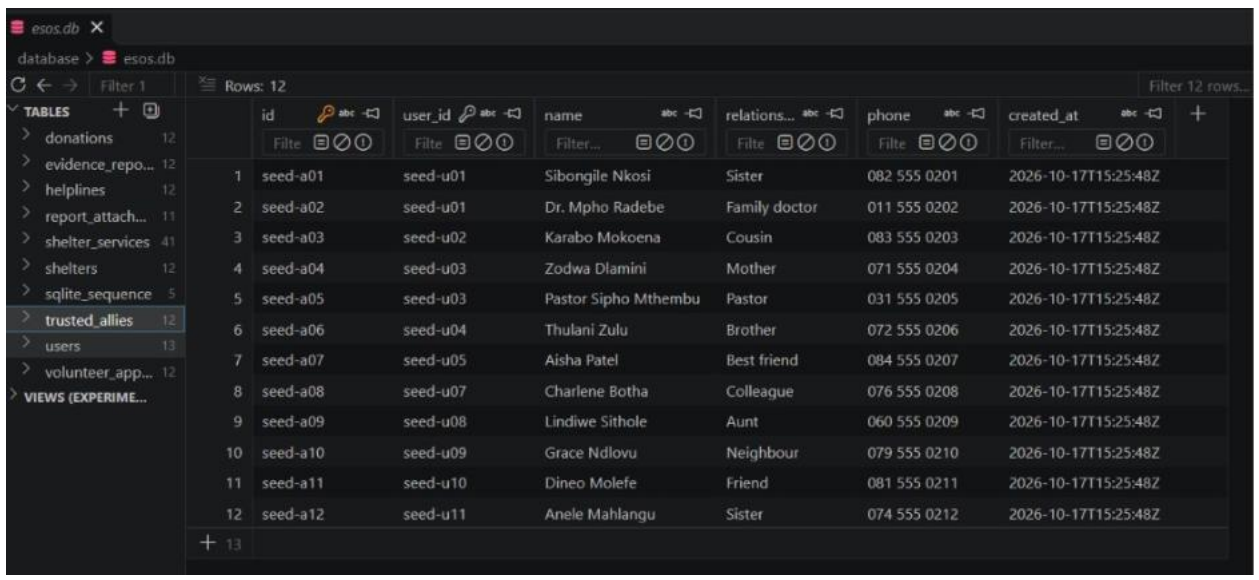


- Each row is one incident report with a title, type, date, time and description.
- `user_id` (`seed-u01` and so on) links each report to its owner in `users`, and the key icon marks it as a foreign key.
- Some users have two reports, which shows one user having many reports.

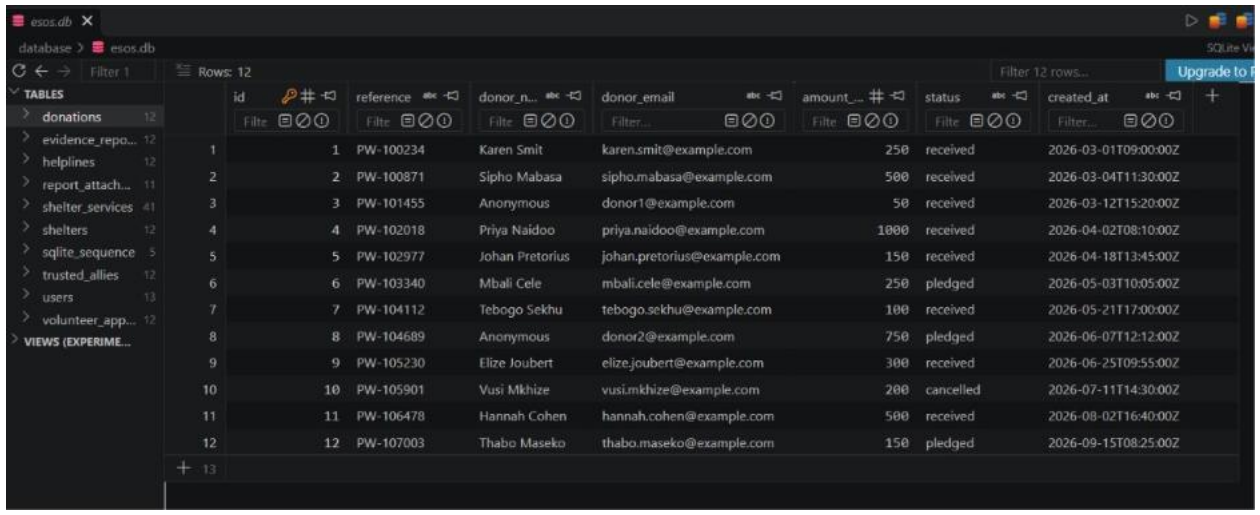


- Each row is one file attached to a report, such as a voice note, photo or PDF. `report_id` is the foreign key to `evidence_reports`.

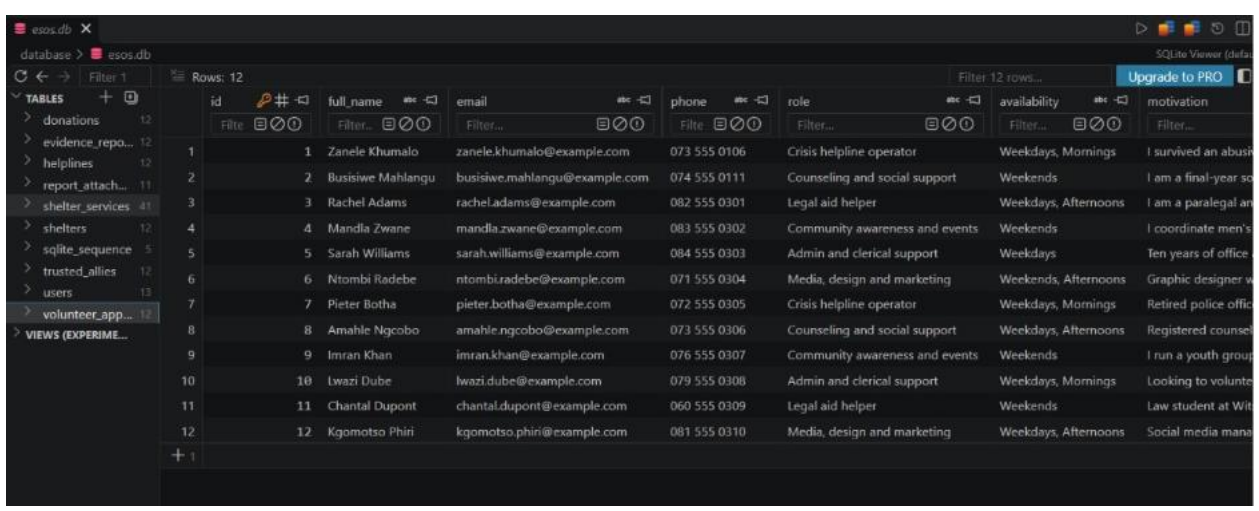
- `seed-r03` and `seed-r06` each have two files, and some reports have none.



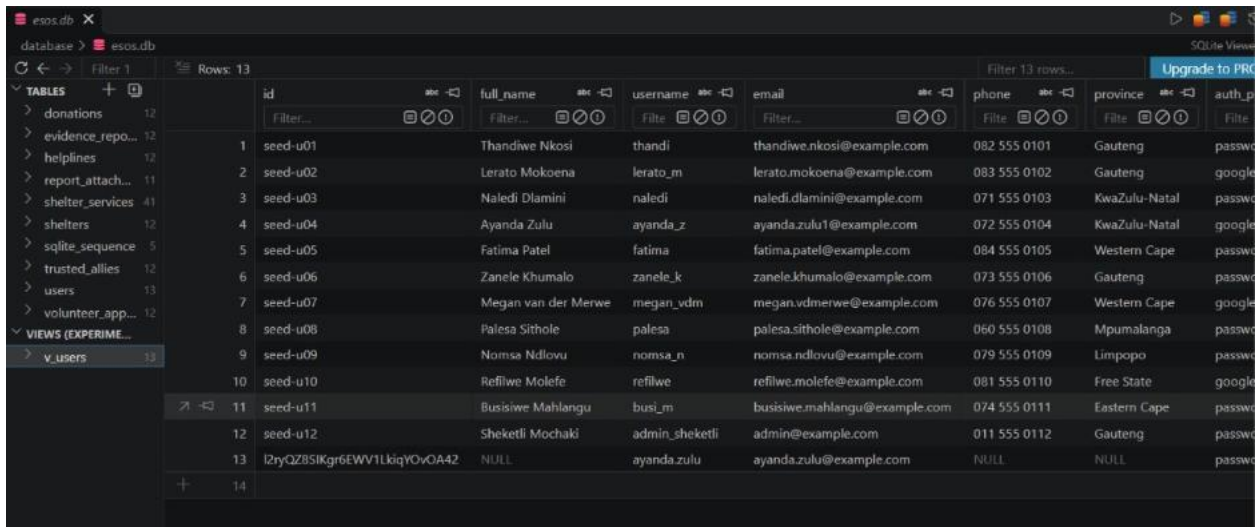
- These are the trusted contacts, each with a name, relationship and phone number.
- `user_id` links each ally to the user who added them. `seed-u01` has two allies.



- Each row is a donation with a unique reference (PW-100234 and so on), a donor, an email, an amount in rand and a status.
- The statuses shown are received, pledged and cancelled, as the CHECK rule allows.



Each row is an applicant with a name, email, phone, the role applied for and availability. The motivation column is cut off on the right.



- The sidebar shows `v_users` under "Views" rather than "Tables", so it holds no data of its own.
- It shows the same 13 users, including the `auth_provider` column (password or Google), because it is a saved query on `users`.

6 DevOps Lifecycle and Automated Quality Assurance

6.1 Kanban Board and Task Management

How the team managed task allocation

The team used **Azure DevOps Boards** in the project "E.SOS Project Software Application" as its agile tracker. Work was split into three levels:

- **Features** are the large capabilities of the system, such as Secure Login, Emergency SOS Alerts and Evidence Management (Figure 4.1d).
- **User stories** describe what a user needs to do, such as "Log in securely" or "Submit an incident report" (Figure 4.1e).
- **Tasks** are the development steps under each story, such as designing a page or setting up pipeline triggers (Figure 4.1a).

Every item has an **owner**. Most items are assigned to Sheketli Motshidisi Mochaki, with other members taking specific stories: Lerato Lizzy Molokomme took the private user dashboard, Mulweli Peggy Mudau took storing and retrieving system records, and Kgaogelo Bridge took the testing task on the Android story. This kept responsibility visible to everyone.

Moving items across the Kanban columns

The board has four columns: **New**, **Active**, **Resolved** and **Closed**.

- An item starts in **New** when it is planned but not started.
- It moves to **Active** when someone begins work.
- It moves to **Resolved** when development is finished and ready for checking.
- It moves to **Closed** when it is complete and accepted.

Figure 4.1b shows an early snapshot. Five stories were New, three were Active, one was Resolved ("Access the private user dashboard") and one was Closed ("Submit an incident report"). Figure 4.1c shows a later snapshot where progress is visible:

- "Manage passwords and account settings" moved from New to Active.
- "Store and retrieve system records" moved to Resolved.
- Active went from 3/5 to 4/5 and Resolved from 1/5 to 2/5.

The numbers beside the Active and Resolved column names are **work-in-progress limits**, which stop too much work being started at once.

Managing timelines

The **Sprints Taskboard** (Figure 4.1a) tracks the tasks inside Iteration 1. It breaks each story into numbered steps. For example, story 17 "Use the Android mobile application" has six tasks, from "Set up the Android application structure" to "Test the application on a supported emulator or device". Tasks the team was working on (103, 105, 107) sit in Active, and tasks still waiting (101, 104, 106, 108) sit in New. The **Backlog** views (Figures 4.1d and 4.1e) hold the ordered list of all features and stories, so the team could decide what to build next.

Screenshots

Figure 4.1a. Sprints Taskboard for Iteration 1, showing the tasks under stories 17 and 16 across the New and Active columns.

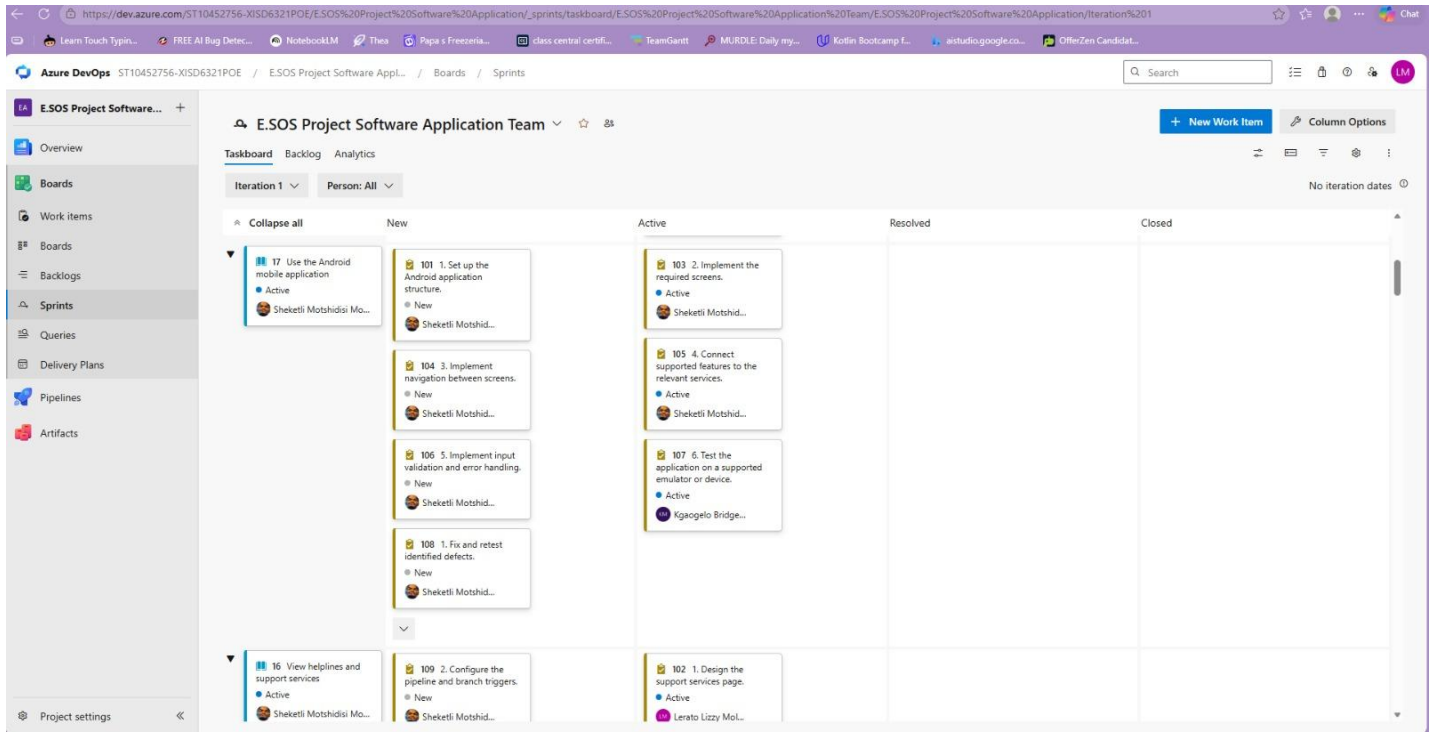


Figure 4.1b. Kanban board, earlier snapshot: Active 3/5 and Resolved 1/5.

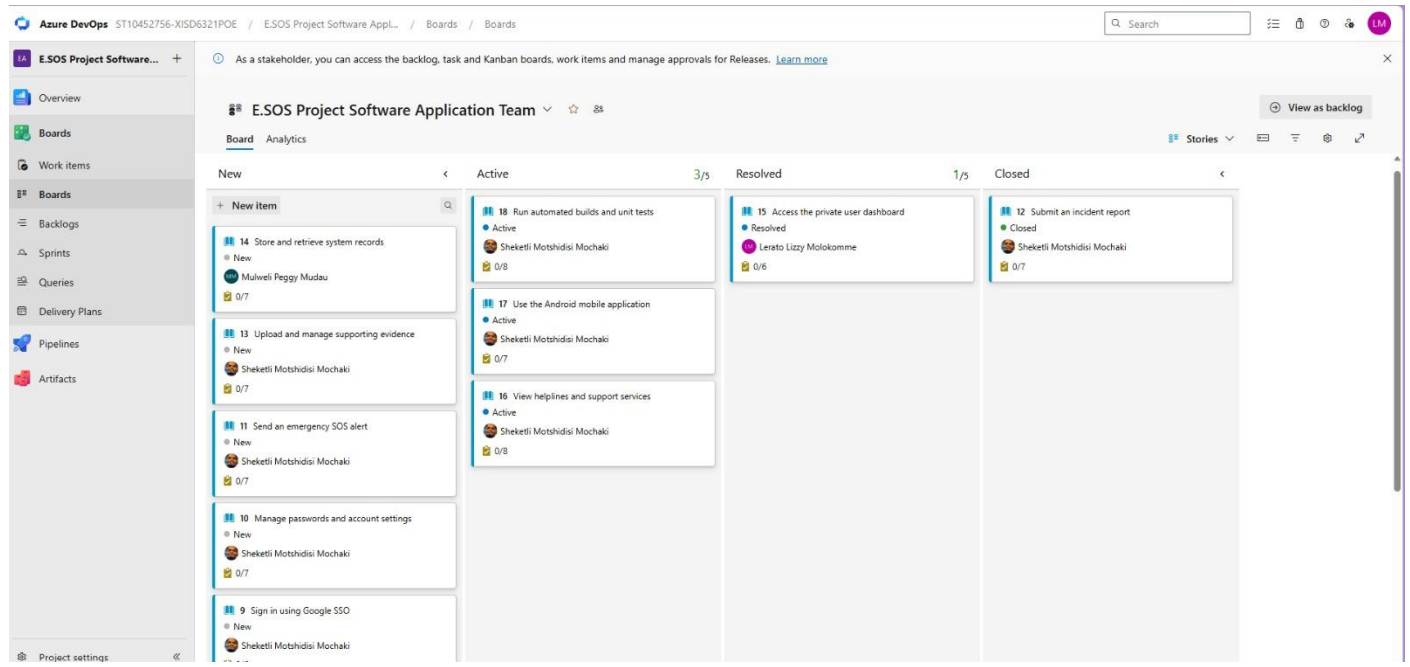


Figure 4.1c. Kanban board, later snapshot: Active 4/5 and Resolved 2/5.

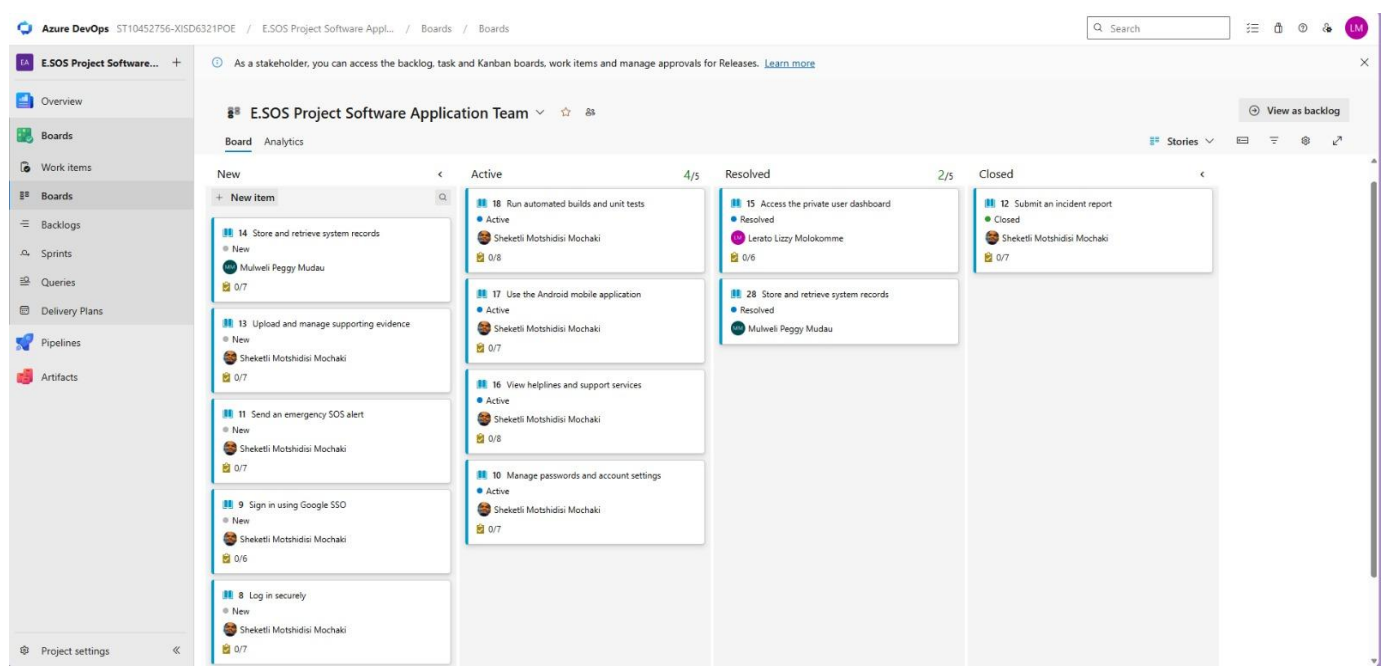


Figure 4.1d. Features backlog: 15 features, F02 to F16.

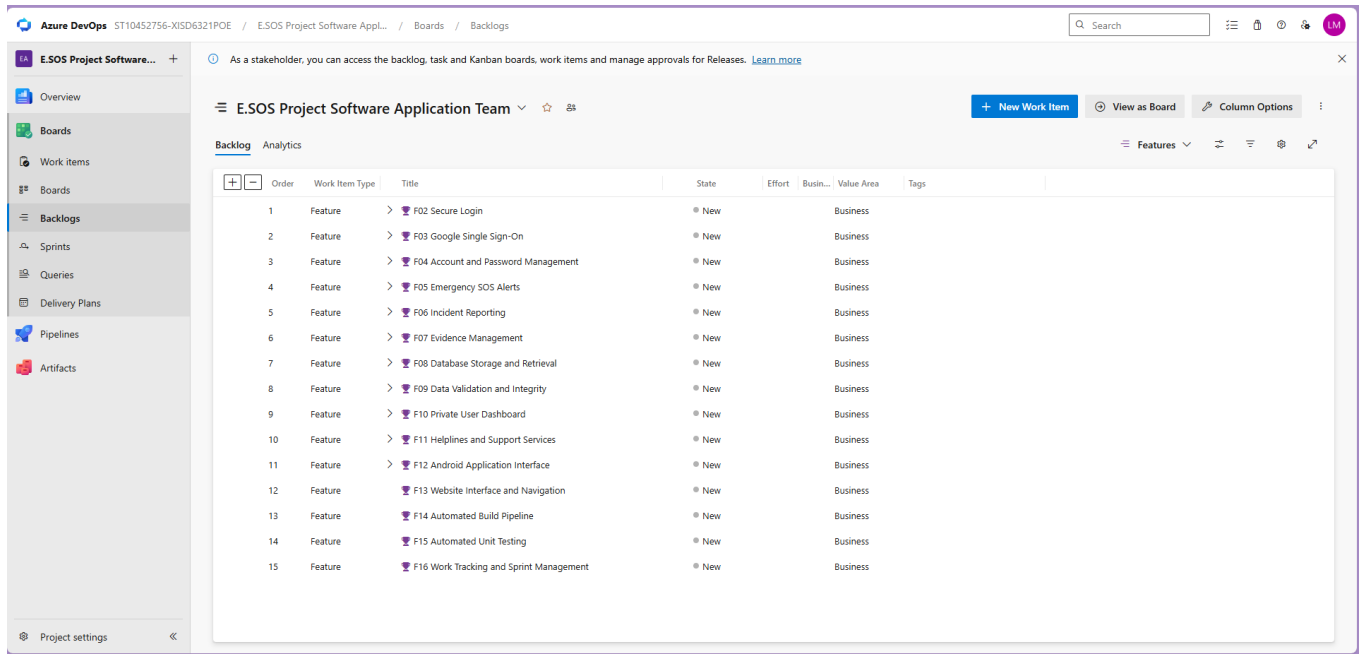
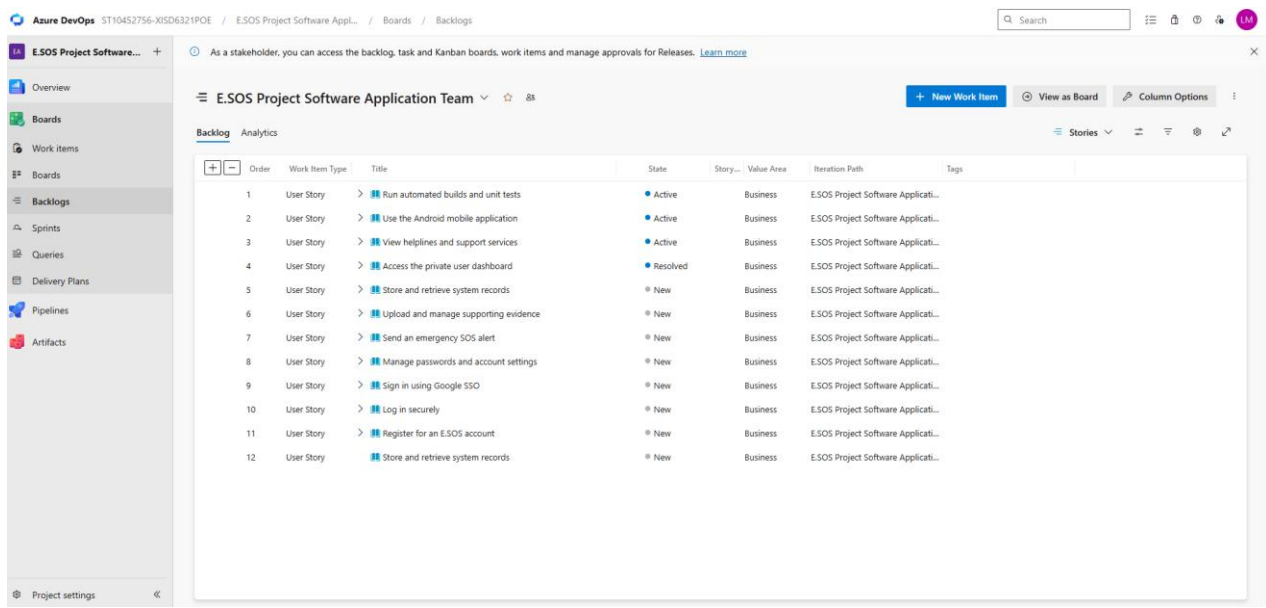
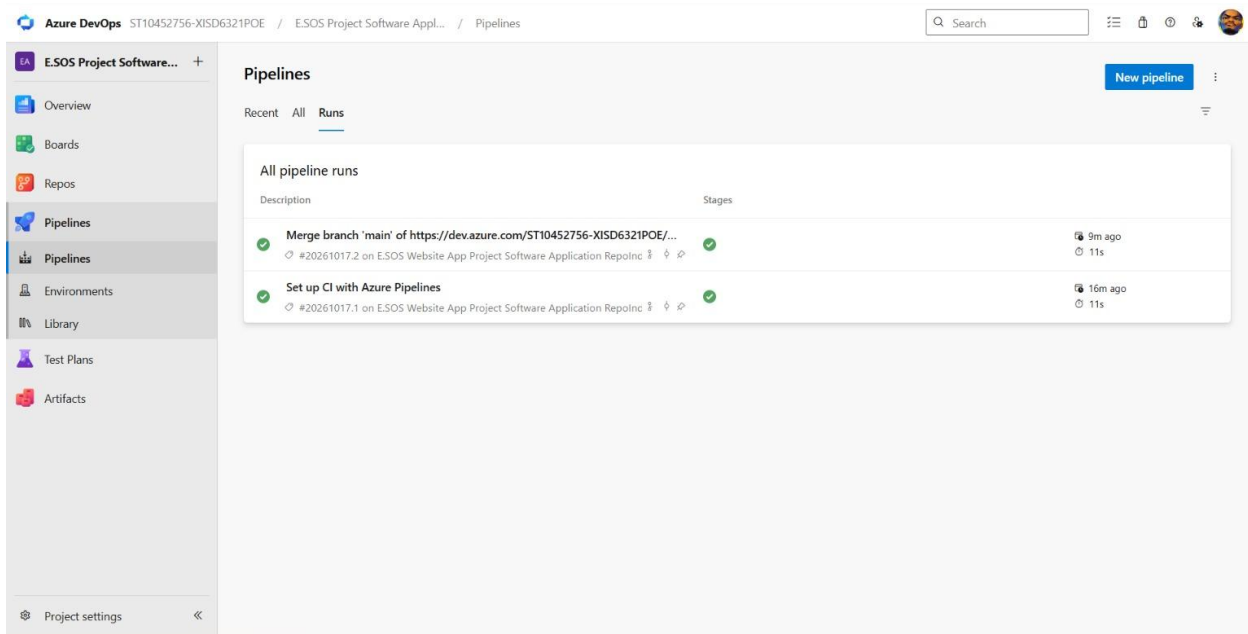


Figure 4.1e. User stories backlog: 12 user stories with state and iteration path.



6.2 Automated Build Pipeline



To automate the deployment workflow, a continuous integration (CI) pipeline was configured in Azure DevOps. The pipeline is linked to the project's main repository branch, so every time updates are pushed or merged into main, a build runs automatically without any manual steps. This ensures that errors introduced by new code are detected early and that the project can always be built successfully. Figure 4 shows the pipeline run history, where both automated executions completed successfully: the initial "Set up CI with Azure Pipelines" run (#20261017.1) and a later run triggered by a merge to main (#20261017.2), each completing in 11 seconds.

6.3 Automated Unit Testing and Build Results

Why automated testing was used

The e.sos platform handles sensitive data such as login credentials, incident evidence and emergency contacts, so a small mistake in validation or security code could lock out a user or expose their data. The team therefore added automated tests that check the most important logic every time the code changes. Each test runs in milliseconds, gives a clear pass or fail, and catches a broken rule before it reaches a user. The tests are written for both clients of the platform: the **web app** and the **Android app**.

Why automated testing was used

The e.sos platform handles sensitive data such as login credentials, incident evidence and emergency contacts, so a small mistake in validation or security code could lock out a user or expose their data. The team therefore added automated tests that check the most important logic every time the code changes. Each test runs in milliseconds, gives a clear pass or fail, and catches a broken rule before it reaches a user. The tests are written for both clients of the platform: the **web app** and the **Android app**.

Web application tests (Vitest)

The web app uses **Vitest**, a test runner for TypeScript. All tests are run with one command, `npm run test:run`, which runs every test once and then exits. The tests sit beside the code they check in `src/lib`, and each file is named after the module it covers:

Android application tests (JUnit and Android Studio)

The Android app (project `E.SOS_App_POWA`) uses **JUnit 4**, run from Android Studio. Its tests are split into two groups, as shown in the project tree in Figure 4.3a:

- **Instrumented tests** (the `androidTest` folder) run on a real or emulated dThe `validation.test.ts` file (Figure 4.3c) covers the **login and sign-up rules**, which are the core of keeping bad data out of the system:
    - **Email checking (`isValidEmail`).** One test group confirms that valid addresses such as a@b.co are accepted, including when they have extra spaces around them. A second group confirms that invalid input is rejected: an empty string, `abc`, `a@b`, an address with a space inside it, and one with no name before the `@`.

- **Form checking (`validateAuth`).** Tests confirm that a correct sign-up and a correct login both pass with no errors. They also confirm that a **username is required on sign-up only**, so a blank username on sign-up produces an error while a blank username on login does not.

The other two test files cover the helper functions in `utils.ts` and the password security code in `crypto.ts`, which fits the platform's use of hashed passwords. Together they protect the code that decides who can register, log in and reach their own records.
evice. The screenshot shows them running on the **Medium Phone emulator**. `DatabaseTest` has a test called `writeUserAndReadInList`, which writes a user into the app's local database and reads it back, so it proves that data is stored and retrieved correctly. `ExampleInstrumentedTest` has a test called `useAppContext`, which checks that the app runs under the expected package name, `com.example.esos_app_powa`.

- **Local unit tests** (the `test` folder) run on the development computer without a device. The project contains `AuthValidatorTest`, `DataParserTest`, `ModelsTest`, `AppTypeConvertersTest`, `SecurityManagerTest` and `ExampleUnitTest`. By their names, these cover login validation, data parsing, the data models, the type converters used by the database, and the security manager.

The run in Figures 4.3a and 4.3b finished with **2 tests, 2 passed** in about 23 seconds. `DatabaseTest` took 98 ms and `ExampleInstrumentedTest` took 7 ms.

Successful build proof

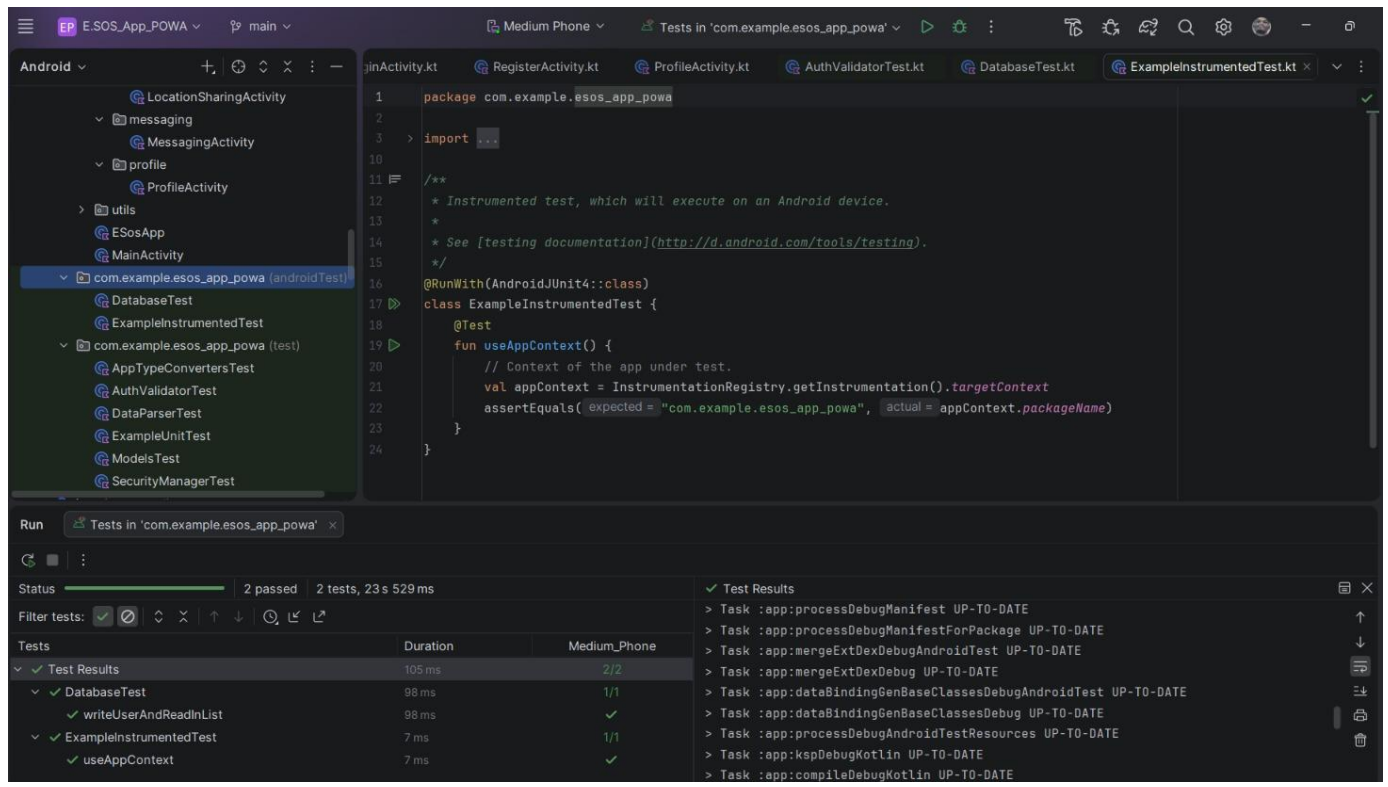
A test run only works if the project compiles first, so each passing run is also proof of a successful build:

- In Android Studio, the Gradle log in Figure 4.3a lists the build steps completed before the tests ran, such as `:app:compileDebugKotlin`, `:app:mergeExtDexDebugAndroidTest` and `:app:processDebugManifest`.
- In the web project, Vitest compiled and ran the TypeScript files and finished with no failures (Figures 4.3c and 4.3d).

In both projects, a failing test would turn the result red and show which rule broke. Running the tests again after every change helps confirm that new work has not broken existing behaviour.

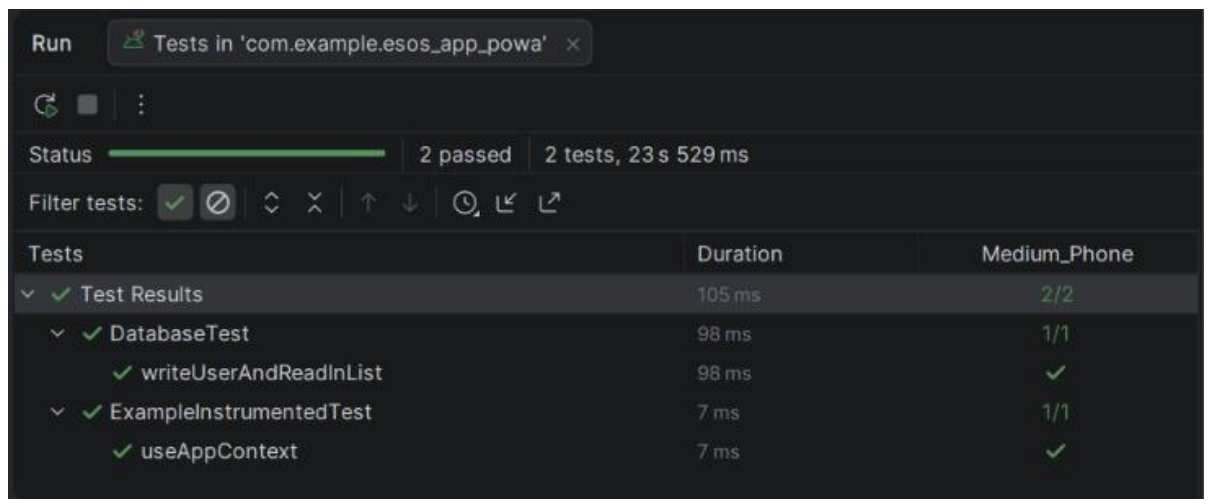
Screenshots

Figure 4.3a. Android Studio: the project tree with the `androidTest` and `test` packages, the `ExampleInstrumentedTest` source, the test results and the Gradle build log.



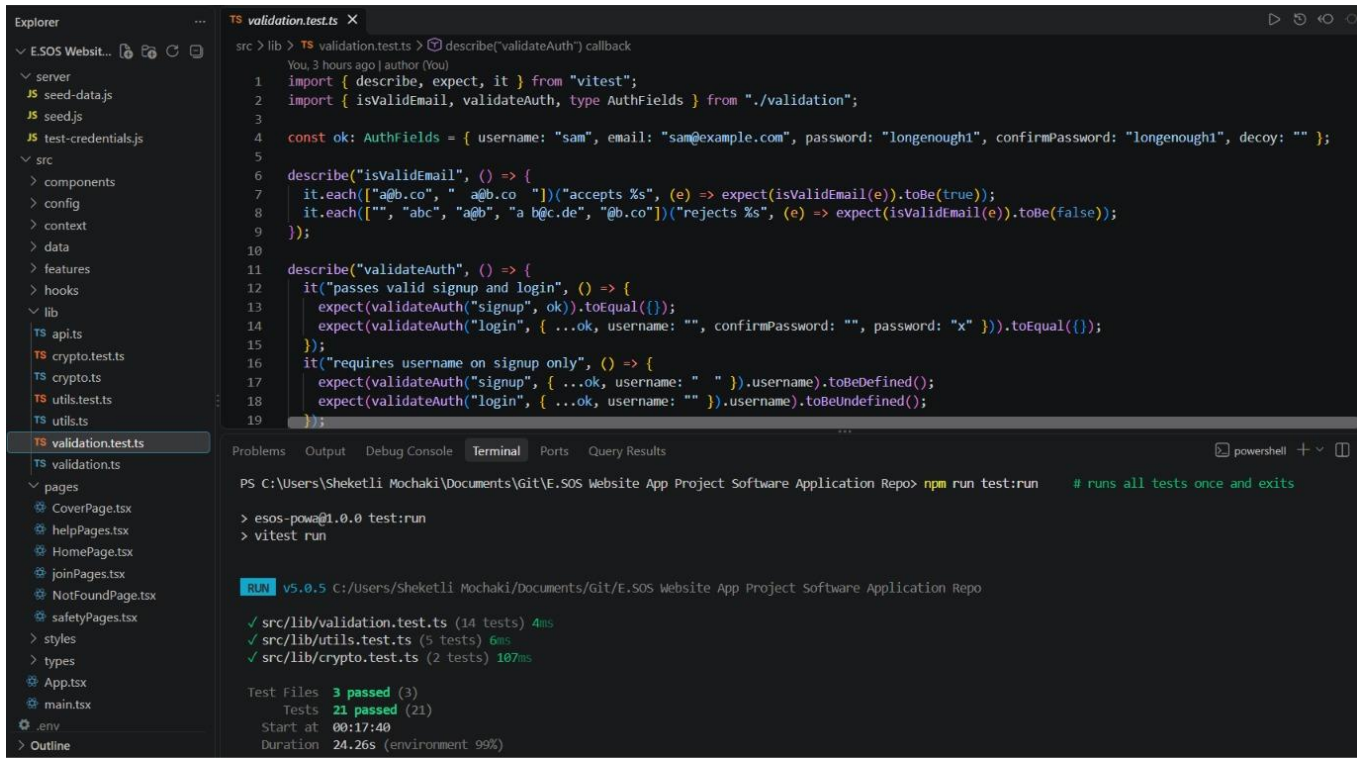
- **Project tree (top left).** This is the Android app project `E.SOS_App_POWA` on the `main` branch. The tree lists the app's screens, such as `MainActivity`, `ProfileActivity`, `MessagingActivity` and `LocationSharingActivity`.
- **Two test folders.** The package marked `(androidTest)` holds the instrumented tests `DatabaseTest` and `ExampleInstrumentedTest`, which run on a device. The package marked `(test)` holds the local unit tests `AppTypeConvertersTest`, `AuthValidatorTest`, `DataParserTest`, `ExampleUnitTest`, `ModelsTest` and `SecurityManagerTest`, which run on the computer.
- **Code editor (top right).** `ExampleInstrumentedTest.kt` is open. The `@RunWith(AndroidJUnit4::class)` line tells JUnit to run the class on an Android device. The `useAppContext` test reads the app's package name and uses `assertEquals` to check that it matches `com.example.esos_app_powa`. If the two differ, the test fails.
- **Toolbar.** The device selector reads "Medium Phone", the emulator the tests ran on, and the run configuration is named after the app's test package.
- **Run panel (bottom left).** The green status bar shows **2 passed, 2 tests, 23 s 529 ms**. The tree below lists `DatabaseTest > writeUserAndReadInList` and `ExampleInstrumentedTest > useAppContext`, each with a green tick.
- **Gradle log (bottom right).** The lines starting with `Task :app:` are the build steps Android Studio ran before the tests, such as processing the manifest, compiling the Kotlin code and merging the test resources. "UP-TO-DATE" means the step had already been completed and nothing had changed, so Gradle skipped repeating it.

Figure 4.3b. Android test results: 2 tests, 2 passed on the Medium Phone emulator.



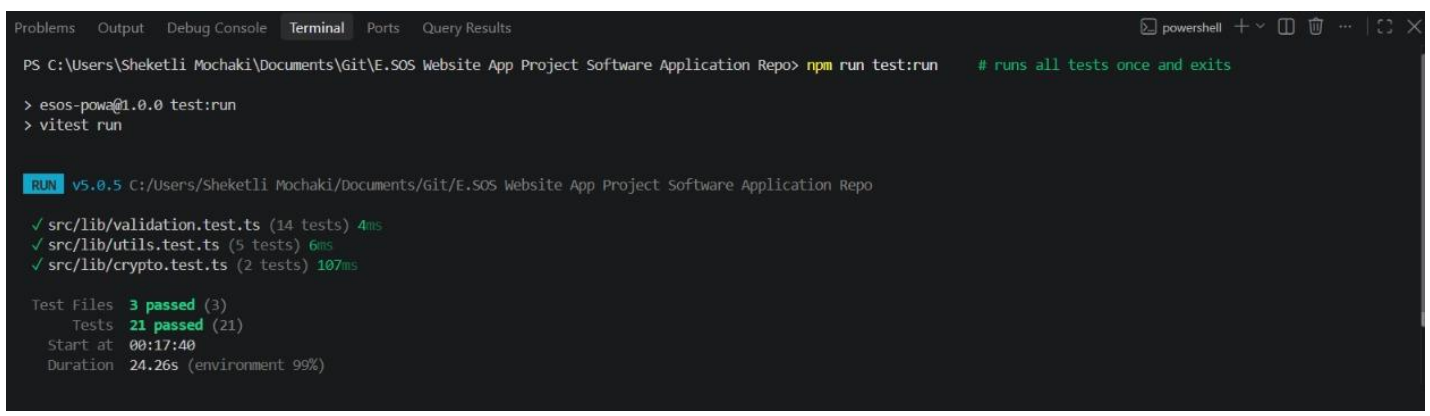
This is a close-up of the Run panel from Figure 4.3a, which makes the numbers easier to read.**Status:** the full green bar means every test passed, with "2 passed" and "2 tests, 23 s 529 ms" as the total.**Test Results tree:** `DatabaseTest` took 98 ms and passed its one test, `writeUserAndReadInList`. `ExampleInstrumentedTest` took 7 ms and passed its one test, `useAppContext`.**Medium_Phone column:** the `2/2` and `1/1` figures show how many tests passed on that emulator out of the total run, so no test was skipped or failed.

Figure 4.3c. VS Code: the `validation.test.ts` source, the `src/lib` test files and the terminal output of `npm run test:run`.



Explorer (left). This is the web project. The `server` folder holds `seed-data.js`, `seed.js` and `test-credentials.js`, which fill the database with test records. The `src/lib` folder holds the app's logic files, and each one sits next to its own test file: `crypto.ts` and `crypto.test.ts`, `utils.ts` and `utils.test.ts`, `validation.ts` and `validation.test.ts`.**Code editor (top right).** `validation.test.ts` is open. It imports the Vitest functions `describe`, `expect` and `it`, and the `isValidEmail` and `validateAuth` functions from `validation.ts`. The ok object near the top is a sample of valid form data (username, email, password, confirm password and an empty decoy password) that the tests reuse.**`isValidEmail` tests.** The first list holds addresses that must be accepted (a@b.co, including one with extra spaces). The second list holds addresses that must be rejected (an empty string, `abc`, a@b, a b@c.de and `@b.co`). Each item in a list becomes its own test.**`validateAuth` tests.** The first test checks that valid sign-up and login data produce no errors. The second checks that a blank username is an error on sign-up but not on login.**Terminal (bottom).** The command `npm run test:run` ran all the tests once. The output underneath is explained in Figure 4.3d.The grey note above the code ("You, 3 hours ago") is a Git annotation from the editor showing who last changed the line and when.

Figure 4.3d. Vitest results: 3 test files and 21 tests passed.



This is a close-up of the terminal from Figure 4.3c.**Command:** `npm run test:run` starts `vitest run`, which runs every test once and then exits, instead of waiting for file changes.**Per-file results:** a green tick means the whole file passed. `validation.test.ts` ran 14 tests in 4 ms, `utils.test.ts` ran 5 tests in 6 ms and `crypto.test.ts` ran 2 tests in 107 ms. The crypto tests are the slowest because password hashing is deliberately heavy work.**Summary:** "Test Files 3 passed (3)" and "Tests 21 passed (21)" mean nothing failed or was skipped. The whole run took 24.26 seconds, which includes starting up the test environment.

Appendix A : Glossary of Terms

Term / Acronym	Definition
AES-256	Advanced Encryption Standard with 256-bit key length. An encryption standard used to protect data at rest.
API	Application Programming Interface. A defined interface through which software systems communicate.
ERD	Entity Relationship Diagram. A visual representation of database entities and their relationships.
GBV	Gender-Based Violence. Violence directed against individuals because of their gender.
GPS	Global Positioning System. A satellite-based navigation system providing real-time location data.
JWT	JSON Web Token. A compact, URL-safe means of representing claims for authentication
MVC	Model-View-Controller. A software design pattern separating application data, user interface, and control logic.
NGO	Non-Governmental Organisation. A non-profit organisation operating independently of government.
OWASP	Open Web Application Security Project. A non-profit organisation producing security guidelines for software development.
POPIA	Protection of Personal Information Act, 2013. South African legislation governing the collection, storage, and processing of personal data.
POWA	People Opposing Women Abuse. A South African non-profit organisation providing support services for survivors of gender-based violence.
RBAC	Role-Based Access Control. A security model restricting system access based on user roles.
SAPS	South African Police Service. South Africa's national law enforcement agency.
SMS	Short Message Service. A text-based communication protocol used for mobile messaging.
SOS	Save Our Souls. An internationally recognised distress signal, adopted as the naming convention for the emergency alert feature.
TLS	Transport Layer Security. A cryptographic protocol providing secure communications over a network.
UML	Unified Modelling Language. A standardised visual modelling language for software system design.
WCAG	Web Content Accessibility Guidelines. An internationally recognised standard for accessible web and mobile content.
WIL	Work Integrated Learning. An academic programme component requiring students to apply learning in a professional practice context.

Appendix B: Technologies Used

Technology	Version / Tier	Purpose in E.SOS App
React Native	Latest stable	Cross-platform mobile application development (Android & iOS)

React.js	Latest stable	Web-based operator dashboard development
Node.js + Express.js	LTS	Backend RESTful API and business logic layer
Firebase Firestore	Cloud	Real-time NoSQL database for live alert management
PostgreSQL	15+	Relational database for structured data and audit logs
AWS S3	Cloud	Encrypted object storage for incident report media attachments
Firebase Auth + JWT	Cloud	User authentication and session management
Firebase Cloud Messaging	Cloud	Push notification delivery to mobile devices
Twilio SMS API	Cloud	Programmatic SMS delivery for offline SOS fallback and contact notifications
Google Maps Platform	Cloud	GPS capture, geocoding, and shareable map link generation
Docker + Kubernetes	Latest	Containerised deployment with auto-scaling and environment consistency
Android Studio	Latest	Android development and emulation environment
GitHub	Cloud	Version control, collaborative development, and CI/CD pipeline

References

Booch, G., Rumbaugh, J. and Jacobson, I. (2005) The Unified Modeling Language User Guide. 2nd edn. Boston: Addison-Wesley.

Date, C.J. (2004) An Introduction to Database Systems. 8th edn. Boston: Pearson Addison Wesley.

Dennis, A., Wixom, B.H. and Roth, R.M. (2012) Systems Analysis and Design. 5th edn. Hoboken: John Wiley & Sons.

Eckerson, W.W. (2010) Performance Dashboards: Measuring, Monitoring, and Managing Your Business. 2nd edn. Hoboken: John Wiley & Sons.

POWA (People Opposing Women Abuse) (2026) About POWA. [online] Available at: https://www.powa.co.za [Accessed: 12 May 2026].

Pressman, R.S. (2014) Software Engineering: A Practitioner's Approach. 8th edn. New York: McGraw-Hill.

Sommerville, I. (2016) Software Engineering. 10th edn. Harlow: Pearson Education.

Stark, J. (2025) Building Safe Digital Systems for Vulnerable Populations. Journal of Human-Computer Interaction, 41(2), pp. 112–134.

Statistics South Africa (2023) Crime Against Women in South Africa: An Analysis of the Victims of Crime Survey and Recorded Crime Data. [online] Available at: https://www.statssa.gov.za/publications/P0341/GPSJS%202023_24%20Final%20P0341%20Victims%20of%20Crime%20Release%20%20.pdf [Accessed: 12 May 2026].

United Nations (2015) Transforming Our World: The 2030 Agenda for Sustainable Development. New York: United Nations.

World Health Organization (2023) Violence Against Women. [online] Available at: https://www.who.int/news-room/fact-sheets/detail/violence-against-women [Accessed: 12 May 2026].

Declaration of Authenticity

I, ___Sheketli Mochaki____ ID Number, ___0201115056081___ hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ____ST10452756_____

STUDENT NAME: ____Sheketli Mochaki____

STUDENT SIGNATURE: ___S.M.M____ DATE: ____23/04/2026_____

I, ______Lerato Molokomme_____ ID Number, _____080805253083_____ hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ____ST104396226_____

STUDENT NAME: ___Lerato Molokommme_____

STUDENT SIGNATURE: ____L.M_____ DATE: _____23/04/2026_____

I, __Mulweli Mudau_____ ID Number, __0504030736087_____ hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: __ST10358894_____

STUDENT NAME: ___Mulweli Mudau_____

STUDENT SIGNATURE: __M.M_____ DATE: __23/04/2026_____

I, ___ Kgaogelo Mashiane ____ ID Number, _____ hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ___ ST10446936_____

STUDENT NAME: ___ Kgaogelo Mashiane _____

STUDENT SIGNATURE: _____K.M_____ DATE: ____23/04/2026_____

Peer Evaluation

(NOT TO BE SHARED WITH TEAM MEMBERS)

Please complete and include the peer-evaluation reports when submitting your final PoE.

GROUP NAME/NUMBER: ___Group 5_____ DATE: ___23/04/2026_____

Please rate each of your team members using the details in the assessment criteria descriptions provided. Marks received from each member of the team will be added up and then averaged for each member's individual mark. Complete a separate evaluation form for each team member.

Name of student being evaluated:	Never	Seldom	Frequently	Always
Kgaogelo Mashiane	0	1	2	3

The student's personal work					
1.	The student contributed good ideas that added value to the project.				3
2.	The student performed their tasks in line with what was expected of them.				3
3.	The student produced high quality work.				3
4.	The student managed their own time well and met deadlines.				3
5.	The student connected their own learning and skills to a real-world problem.				3
The student's work as part of a team (when relevant)					
6.	The student accepted responsibility for a fair portion of the tasks.				3
7.	The student was an enthusiastic member of my team.				3
8.	The student helped others to be successful.				3
9.	The student worked well with other members of the team.				3
10.	The student showcased respect and dignity towards all members of the team.				3
TOTAL:					30/30

Comments:

Excellent work and great contribution to the team.

_____

_____

_____

_____

NAME: ___Mulweli Mudau_____ STUDENT NUMBER: _____ST10358894_____

SIGNATURE: ___M.M_____

(NOT TO BE SHARED WITH TEAM MEMBERS)

Please complete and include the peer-evaluation reports when submitting your final PoE.

GROUP NAME/NUMBER: __Group 5_____ DATE: _____23/04/2026_____

Please rate each of your team members using the details in the assessment criteria descriptions provided. Marks received from each member of the team will be added up and then averaged for each member's individual mark. Complete a separate evaluation form for each team member.

Name of student being evaluated: Lerato Molokomme	Never	Seldom	Frequently	Always

		0	1	2	3
The student's personal work					
1.	The student contributed good ideas that added value to the project.				3
2.	The student performed their tasks in line with what was expected of them.				3
3.	The student produced high quality work.				3
4.	The student managed their own time well and met deadlines.				3
5.	The student connected their own learning and skills to a real-world problem.				3
The student's work as part of a team (when relevant)					
6.	The student accepted responsibility for a fair portion of the tasks.				3
7.	The student was an enthusiastic member of my team.				3
8.	The student helped others to be successful.				3
9.	The student worked well with other members of the team.				3
10.	The student showcased respect and dignity towards all members of the team.				3
TOTAL:					30/30

Comments: Excellent work and great contribution to the team.

_____

_____

_____

_____

NAME: ____Sheketli Mochaki_____ STUDENT NUMBER: _____ST10452756_____

SIGNATURE: _____S.M.M_____

(NOT TO BE SHARED WITH TEAM MEMBERS)

Please complete and include the peer-evaluation reports when submitting your final PoE.

GROUP NAME/NUMBER: ___Group 5_____ DATE: ____23/04/2026_____

Please rate each of your team members using the details in the assessment criteria descriptions provided. Marks received from each member of the team will be added up and then averaged for each member's individual mark. Complete a separate evaluation form for each team member.

Name of student being evaluated:	Never	Seldom	Frequently	Always
Sheketli Mochaki	0	1	2	3

The student's personal work					
1.	The student contributed good ideas that added value to the project.				3
2.	The student performed their tasks in line with what was expected of them.				3
3.	The student produced high quality work.				3
4.	The student managed their own time well and met deadlines.				3
5.	The student connected their own learning and skills to a real-world problem.				3
The student's work as part of a team (when relevant)					
6.	The student accepted responsibility for a fair portion of the tasks.				3
7.	The student was an enthusiastic member of my team.				3
8.	The student helped others to be successful.				3
9.	The student worked well with other members of the team.				3
10.	The student showcased respect and dignity towards all members of the team.				3
TOTAL:					30/30

Comments: Excellent work and great contribution to the team.

_____

_____

_____

_____

NAME: _____ Lerato Molokomme _____ STUDENT NUMBER: _____ST10439626 _____

SIGNATURE: _____L.M_____

(NOT TO BE SHARED WITH TEAM MEMBERS)

Please complete and include the peer-evaluation reports when submitting your final PoE.

GROUP NAME/NUMBER: ___Group 5_____ DATE: ___23/04/2026_____

Please rate each of your team members using the details in the assessment criteria descriptions provided. Marks received from each member of the team will be added up and then averaged for each member's individual mark. Complete a separate evaluation form for each team member.

Name of student being evaluated:	Never	Seldom	Frequently	Always
Mulweli Mudau	0	1	2	3
The student's personal work				

1.	The student contributed good ideas that added value to the project.				3
2.	The student performed their tasks in line with what was expected of them.				3
3.	The student produced high quality work.				3
4.	The student managed their own time well and met deadlines.				3
5.	The student connected their own learning and skills to a real-world problem.				3
The student's work as part of a team (when relevant)					
6.	The student accepted responsibility for a fair portion of the tasks.				3
7.	The student was an enthusiastic member of my team.				3
8.	The student helped others to be successful.				3
9.	The student worked well with other members of the team.				3
10.	The student showcased respect and dignity towards all members of the team.				3
TOTAL:					30/30

Comments: Excellent work and great contribution to the team.

_____
_____
_____
_____

NAME: _____ Kgaogelo Mashiane _____ STUDENT NUMBER: _____

SIGNATURE: _____K.M_____

Self-Reflection Report

Mudau Mulweli

ST10358894

Introduction

Work Integrated Learning (WIL) plays an important role in bridging the gap between academic knowledge and real-world application. Through this project, I was able to apply theoretical concepts to a practical scenario, which helped me gain a deeper understanding of how systems are designed, planned, and implemented. The WIL project also exposed me to teamwork, problem-solving, and professional communication, all of which are essential skills in the IT and business environment. Overall, this experience helped me grow both academically and personally.

Skills Learnt

During this WIL project, I developed and applied several important skills across different areas:

Industry-Specific Skills

I improved my **report writing and documentation skills**, especially when working on structured sections like the project overview, budget estimates, and system requirements. I also learned how to organise information professionally and follow academic formats such as Harvard referencing. Additionally, I gained experience in **planning and structuring a system**, including understanding how features, users, and stakeholders interact.

Interpersonal Communication Skills

Working in a group helped me develop my **communication skills**, especially during discussions, brainstorming sessions, and decision-making. I learned how to listen to others' ideas, share my own input, and provide feedback respectfully. Group meetings required clear communication to ensure everyone understood their tasks and responsibilities.

Management Skills

I developed **time management skills** by meeting deadlines and balancing different parts of the project. I also improved my ability to handle pressure when tasks were due at the same time. In situations where problems occurred, such as confusion or delays, I applied basic **problem-solving and crisis management skills** to find solutions and keep the work progressing.

Role in the Team

During the WIL project, I worked as part of a team where each member had specific responsibilities. I contributed mainly to sections such as the **Project and Product Overview** and **Budget Estimate, Project plan** ensuring that my work aligned with the rest of the group document.

Leadership and Instructions

Although I was not the main leader, I followed instructions given by the group leader and contributed ideas during discussions. I also took responsibility for completing my assigned tasks on time and ensuring quality work.

Contribution to Team Success

My contribution helped ensure that our document was complete, well-structured, and met the required academic standards. I made sure my sections were detailed, clear, and aligned with the project objectives.

Group Dynamic

The group dynamic was generally positive. We worked together, shared ideas, and supported each other when needed. Everyone had different strengths, which helped improve the overall quality of the project.

Dealing with Concerns and Conflict

There were moments where misunderstandings or differences in opinions occurred. These were handled through discussion and compromise. We focused on the goal of completing the project successfully rather than personal disagreements.

Research, Technology and Presentation of Information

During this project, I was required to find relevant and useful information to complete my tasks.

Scenario 1

I had to research information about **POWA (People Opposing Women Abuse)** to understand their services and how the system could support their work.

- **Where I found information:** Official POWA website and online articles
- **Technology used:** Google search, web browser
- **Presentation:** I used this information in written paragraphs within the report using proper academic structure and referencing

Scenario 2

I had to research **budget estimates and system features** for the application.

- **Where I found information:** Online resources, examples of similar systems, and academic materials
- **Technology used:** Internet research tools and Microsoft Word
- **Presentation:** Information was presented in both paragraph form and tables (e.g., budget table and feature table)

This process helped me understand the importance of selecting reliable sources and presenting information clearly.

Personal Strengths and Weaknesses

Tasks I Did Well

- Writing structured and detailed sections of the report
- Researching relevant information for the project
- Following academic formatting and referencing
- Meeting deadlines for my assigned tasks
- Contributing ideas during group discussions

My Strengths

1. Good research skills
2. Ability to organise and present information clearly
3. Strong written communication
4. Ability to work well in a team
5. Time management and meeting deadlines

Tasks I Did Not Do Well

- Sometimes I took longer to complete tasks due to overthinking
- I was not always confident when sharing ideas in the group

Reasons for Challenges

These challenges were mainly due to a lack of confidence and wanting my work to be perfect before submitting it.

How I Can Improve

- Improve confidence by participating more in discussions
- Manage time better by avoiding overthinking tasks
- Practice faster decision-making when working under pressure

Stakeholder Relationship

During the project, the lecturer acted as a guide and evaluator. The relationship was professional, and the lecturer provided clear instructions and feedback when needed. This helped the team stay on track and understand the expectations of the project.

Within the team, relationships were cooperative and respectful. Communication was important in ensuring that everyone contributed and understood their roles. Overall, the teamwork experience helped me understand the importance of collaboration in real-world projects.

Sheketli Mochaki

ST10452756

Introduction

Work Integrated Learning (WIL) gave me the opportunity to move from studying software development concepts in theory to applying them on a live, socially significant project. As Lead Developer on the Emergency SOS Services Application (E.SOS App), developed in partnership with POWA (People Opposing Women Abuse), I was responsible for the technical backbone of the project across both the planning stage and the requirement analysis and system design stage: assessing technical feasibility, defining the system architecture, designing the database, and embedding security into the platform from the ground up.

Working on a safety-critical application for survivors of gender-based violence sharpened my appreciation for the weight that technical decisions can carry. A poor architecture choice or an unverified assumption in this kind of system is not just a bug — it could directly affect someone's safety. That responsibility shaped how I approached every technical task across the project, from the earliest feasibility assessment in the project charter through to the detailed database and security design in the system design document.

Skills Learnt

Industry-Specific Skills

During the charter phase, I contributed to the Technical Feasibility section, evaluating hardware and software availability, and proposing mitigation strategies such as multi-source location triangulation (GPS, Wi-Fi, and cellular data) and an SMS-based fallback using the Twilio API for offline SOS functionality. This required me to think beyond ‘what works in theory’ and consider real deployment constraints — device compatibility, network reliability in under-resourced areas, and the cost implications of cloud infrastructure choices. I also contributed technical architecture input to the Work Breakdown Structure, helping define the backend, API, and cloud infrastructure work packages, and assisted in identifying technical risks such as third-party API failure and performance degradation under concurrent load, both of which fed into the project's risk register.

In the requirement analysis and system design phase, I translated the use cases into a class diagram, defining the User, SOSAlert, IncidentReport, EmergencyContact, and POWAOperator classes along with their relationships (composition, aggregation, association, and dependency). This required me to think carefully about ownership and lifecycle — for example, deciding that an SOSAlert is a composition of User (it cannot exist without an owning user) while EmergencyContact is an aggregation (a contact can exist independently of any specific alert event). I also led the database design, producing the normalised table structures for users, sos_alerts, incident_reports, emergency_contacts, and messages, and the accompanying entity relationship diagram. Designing for POPIA compliance from the outset — deciding which fields needed encryption at rest, applying data minimisation, and defining retention-relevant timestamps — gave me hands-on experience applying data protection law to a real schema rather than just reading about it. On the architecture side, I helped define the three-tier MVC structure (React Native/React.js presentation layer, Node.js/Express application layer, Firebase Firestore and PostgreSQL data layer) and justified each technology choice against the project's actual constraints, such as needing offline-capable real-time updates (Firestore) alongside relational integrity for audit logs (PostgreSQL).

Interpersonal Communication Skills

Translating technical constraints into language the rest of the team and our client could understand was a skill I had to actively develop. Explaining why certain technology choices mattered — for example, why an SMS fallback was non-negotiable for a GBV support tool — required me to communicate technical reasoning in terms of user impact rather than implementation detail. Presenting the class diagram and ERD to the team later in the project required me to explain object-oriented and relational modelling concepts to teammates with different technical backgrounds. I learned to walk through diagrams incrementally — starting with the core entities before introducing relationship types — so that design reviews were genuinely collaborative rather than a one-way technical briefing.

Management Skills

I managed my own technical workstream independently across both phases: scoping the feasibility assessment, prioritising which technical risks needed mitigation strategies before development could begin, and later sequencing the dependencies between my design tasks and the work of the UI/QA Engineer and Business Analyst, since the database schema and API contracts needed to be stable before wireframing and interaction-flow design could be finalised. This taught me to plan technical deliverables deliberately, flagging which design decisions were blocking other people's work rather than letting bottlenecks surface late.

Role in the Team

Leadership and Responsibilities

As Lead Developer, I held primary responsibility (Accountable/Responsible in the RACI matrix) for System Architecture Design, Database Design, Backend/API Development, and Security Implementation throughout the project. During the charter phase this meant assessing whether the team's proposed technology stack, hardware, and third-party services (Google Maps, Twilio, Firebase, AWS) were realistic given our skill levels, budget, and timeline, and flagging technical risks early rather than leaving them as open concerns. In the system design phase, this translated into concrete artefacts: the high-level architecture table, the technology stack with justifications, the security architecture (TLS 1.3, AES-256, JWT with refresh rotation, RBAC, OWASP-aligned input validation), the class diagram, and the full database schema and ER diagram.

Contribution to Team Success

By flagging feasibility issues and proposing mitigations during the charter stage, rather than discovering them mid-development, I helped the team avoid the kind of late-stage technical surprises that derail project timelines. Later, by producing a stable, requirement-traceable architecture and schema, I gave the rest of the team a firm technical foundation to design around. The UI/QA Engineer's wireframes and the Business Analyst's use-case documentation could both reference concrete data structures rather than placeholders, which kept the project's technical and design work internally consistent.

Group Dynamic

The team worked cooperatively, with each member's specialisation respected. As the team member with the deepest technical background, I was often consulted on feasibility questions raised by other sections of the project, such as budget estimates for backend and cloud services, and design reviews later on were collaborative: feedback from the Business Analyst on use case completeness directly shaped which entities and relationships I included in the class diagram, and accessibility requirements from the UI/QA Engineer influenced how I structured user-facing data fields, such as language preference and notification settings.

Dealing with Concerns and Conflict

There were occasional disagreements about the appropriate level of technical detail and direction — for example, whether to treat SAPS system integration as guaranteed functionality (I advocated for treating it as a 'Could Have' given uncertain partnership timelines), and later whether to use a single database technology or the hybrid Firestore/PostgreSQL approach we ultimately adopted. I argued for the hybrid model because real-time alert data and structured audit/compliance data have genuinely different access patterns. Resolving both disagreements through a clear comparison of trade-offs, rather than personal preference, was a useful exercise in grounded technical decision-making.

Research, Technology and Presentation of Information

Scenario 1: Technology Stack and Feasibility Research

- Where I found information: official documentation for Google Maps Platform, Twilio, Firebase, and AWS; comparative articles on React Native vs Flutter for cross-platform development.
- Technology used: Google, official API documentation, GitHub, Android Studio.
- Presentation: Findings were distilled into the Technical Feasibility Issues table, pairing each technical risk with a concrete resolution/mitigation strategy rather than a general statement.

Scenario 2: Architecture, Security and Database Research

- Where I found information: OWASP Top 10 documentation, POPIA legislation, database design textbooks, UML class diagram conventions, and official PostgreSQL/Firestore documentation.
- Technology used: Microsoft Word, Microsoft Project, diagramming software, internet research tools.
- Presentation: Information was structured into the WBS technical streams and risk register as tables, and later into the Security Architecture section and database schema/ERD, with visual diagrams used wherever a technical audience needed a precise, scannable specification rather than narrative text.

This process reinforced the importance of citing authoritative technical sources (official API and legislative documentation over forum posts) when making architecture and data-handling decisions that the whole team and the client would rely on.

Personal Strengths and Weaknesses

Tasks I Did Well

- Assessing technical feasibility realistically rather than optimistically
- Designing a normalised, POPIA-aware database schema
- Producing a class diagram with correctly justified relationship types
- Selecting and justifying a technology stack against real project constraints
- Embedding security requirements directly into the architecture rather than treating them as an afterthought

My Strengths

1. Strong technical problem-solving and systems thinking
2. Security-conscious design thinking
3. Ability to justify technical decisions with clear reasoning
4. Comfortable researching and validating third-party tools before committing to them
5. Effective at sequencing technical dependencies for the team

Tasks I Did Not Do Well

- I sometimes focused too narrowly on technical correctness and under-estimated how long it would take to explain those decisions to non-technical stakeholders
- I initially under-documented some of the rationale behind class relationship choices, which meant I had to revisit and clarify them during review
- I could have involved the rest of the team earlier in reviewing the database schema, rather than presenting a more finished version

Reasons for Challenges

These challenges stemmed from genuinely wanting the technical plan and design to be sound before sharing them, which meant I sometimes spent longer than planned validating assumptions or working through artefacts independently before bringing them to the team.

How I Can Improve

- Timebox feasibility and design research more tightly and flag open questions rather than resolving everything alone before moving on
- Share draft technical artefacts earlier, even when incomplete, to get feedback sooner
- Document design rationale alongside diagrams themselves, not as a separate afterthought
- Continue building confidence in presenting technical trade-offs persuasively to non-technical stakeholders

Stakeholder Relationship

My key stakeholders throughout the project were my own teammates, who depended on a stable feasibility assessment, architecture, and schema to plan and design their own sections, and indirectly POWA and its operators, whose operational needs — reliability, offline access, low-cost sustainability, and survivor confidentiality — shaped every technical decision I made. I made sure technical recommendations were explained in terms of what they meant for POWA's day-to-day use of the system, and kept every design decision traceable back to the documented functional and non-functional requirements, so the system we designed would genuinely serve POWA's needs rather than just being technically elegant.

Conclusion

Working as Lead Developer on the E.SOS App project taught me that technical leadership starts well before any code is written — it begins with honest feasibility assessment and carries through into every architecture, data, and security decision that follows. Designing systems intended to support survivors of gender-based violence gave real weight to decisions I might otherwise have treated as purely technical, such as offline fallback support, data protection compliance, and user safety. This experience has strengthened my systems design ability, my security awareness, and my confidence in justifying technical decisions to a broader team, and has prepared me to take on more senior technical leadership responsibility in future professional environments.